Smart Clinic Appointment & Patient Management System (SCAPMS)

COS202 Programming II — Lab Assessment Project

Smart Clinic Appointment & Patient Management System (SCAPMS)

C++17 · Qt 6.5.3 · Qt Widgets

Student: OlisaEmeka Davies Chinedu-Eze

Date: 13 November 2026

About this document. The assessment asks for a single document that holds the source code under four headings, the screenshots, a UML diagram and a two to three page report. This file is that document, in the order the brief lists it: **1** the report, **2** the six screenshots, **3** the UML diagrams, **4** the source code, **5** the data files, and **6** the readme and the detailed supplementary report.

Source code. Every file of the program is listed in section 4 of this document. The same code is also kept in the repository https://github.com/OlisaDavies/COS-202-LAB-ASSESSMENT, which holds the build scripts, the project file and the screenshots in their original pixel size.

Contents

1. Project report
2. Screenshots of the graphical interface
3. UML class diagrams
4. Source code
    1. Header files
    2. Source files
    3. GUI files and resources
    4. Data files required to run the application
5. Readme and the detailed supplementary report

1. Project Report

Academic Project Report

Course: COS202 Programming II – Lab Assessment Project **Project:** Smart Clinic Appointment & Patient Management System (SCAPMS) **Student:** OlisaEmeka Davies Chinedu-Eze **Language & Framework:** C++17, Qt 6.5.3 (Qt Widgets) **Build Environment:** Visual Studio 2022 MSVC Build Tools, qmake

---

1. Project Description

A clinic running on paper records suffers double bookings, a waiting room nobody can track, and deleted records with no way back. SCAPMS is a desktop application that fixes those problems. It manages patients in three categories — general, emergency and follow-up — and lets a member of staff register, edit and delete a record; book, cancel and complete an appointment; manage a first-in first-out waiting queue; search and sort the records with the algorithm of their choice; undo the last administrative step; and read four clinic reports. All patient and appointment data is written to and read from CSV files, so the clinic survives between runs.

2. System Features

Feature	Where it lives in the interface
Register, edit and delete a patient	Patient Records panel
Schedule, cancel and complete an appointment	Appointments panel
Waiting room: add a patient, call the next one	Waiting Queue panel
Search by ID, keyword, name or category; sort by ID, name or age	Search & Sort panel
Undo the last change to a patient or an appointment	Administration panel, or Ctrl+Z
Save and load the clinic files	Administration panel, or Ctrl+S / Ctrl+O
Four clinic reports	Clinic Reports panel
Live dashboard and the recently accessed records	Administration panel

3. Class Hierarchy

The core of the model is the **abstract base class** `PatientRecord`, which holds the details every patient has in common: patient ID, name, age, contact number and category. It declares four pure virtual functions — `getTypeLabel()`, `getPatientReport()`, `getSpecialDetail()` and `clone()` — so it can never be created on its own. It carries concrete methods that every category shares, and its destructor is virtual, so deleting through a `PatientRecord*` frees the subclass.

Three subclasses give each category its own behaviour: `GeneralPatient` (a standard outpatient visit with a consultation fee), `EmergencyPatient` (an urgent case with a severity level 1–3 and a triage note), and `FollowUpPatient` (a returning patient with the date of the last visit and the assigned doctor).

Interface. C++ has no `interface` keyword, so **`Appointable`** is a pure abstract class that only declares `scheduleAppointment()`, `cancelAppointment()` (with and without an ID), `hasScheduledAppointment()` and `getAppointmentDetails()`. It holds no data, so any class that can be booked may implement it. `EmergencyPatient` deliberately does not: a trauma case is never booked in advance, and because the database routes every booking through `Appointable`, the rule is enforced by the type system rather than by a check on the category name. A second interface, **`Billable`**, is implemented at the same time through the intermediate abstract class `ScheduledPatient`, showing that a class may implement more than one interface.

Composition and association. A patient owns its appointments: `PatientRecord` holds a `std::vector<Appointment>`, so a patient is associated with one or more appointments and they are created and destroyed together. `ClinicDatabase` owns the records in a `std::vector<std::unique_ptr<PatientRecord>>`, so each record is freed exactly once.

Encapsulation. Every patient field is private and reached through an accessor. The rules of the clinic — a valid age, a unique ID, a date in the right format — are checked in the controller layer, so the screens and the data can never disagree about them.

4. Data Structures Used

Structure	Where	Why this one
`std::vector`	the patient roster; the appointments of each patient	O(1) append, and the record list grows and shrinks all day
`std::queue`	the consultation waiting room	First-in first-out: the patient who joined first is served first
`std::stack`	the administrative undo history	Last-in first-out, exactly what "undo the last step" needs
`PatientRecord* recentPatients[10]`	the recently accessed cache	A fixed-size array of ten slots, held oldest at the front and newest at the back. Using a record moves it to the newest place; when it is full the record used longest ago makes room

5. Searching Algorithms Used

Every search is written by hand; the program never calls `std::find` or `std::binary_search`.

1. **Linear search (O(n))** — `linearSearchByID` scans the roster and returns the position of the patient whose ID is *exactly* the text typed, in a list of any order.
2. **Linear keyword search (O(n))** — `linearSearchByText` finds any patient whose ID *or* name holds the text, ignoring letter case, so "GEN" finds every general outpatient.
3. **Binary search (O(log n))** — `binarySearchByID` halves the search space on each step. It needs a sorted list, so the panel makes a sorted copy before calling it, which guarantees the precondition.
4. **Recursive search** — `recursiveSearchByName` calls itself with the next position until it runs off the end of the list, collecting every patient whose name holds the text.
5. **Category search (O(n))** — `searchByCategory` returns every record of one category.

The routine is chosen from a combo box, so the search that runs matches the state of the data: binary search for a sorted list, linear search otherwise.

6. Sorting Algorithms Used

Both sorts are written by hand and chosen from a combo box.

1. **Selection sort ($O(n^2)$)** — finds the smallest remaining record and performs one swap per pass, so it does at most n − 1 swaps. It sorts in place and needs no extra memory, which suits the short lists of a clinic.
2. **Merge sort (O(n log n))** — splits the list in half and rejoins it. The merge takes the record of the left half first unless the record of the right half is strictly smaller, which makes the sort **stable**: records with the same key keep the order they had.

Records are ordered by patient ID, patient name or patient age. Names and IDs are compared without regard to letter case, so a name typed in small letters sits in its proper place rather than at the end. A label under the sort controls explains the chosen routine and why it fits, and the sort and the binary search share one comparison rule, so the list stays sorted the way the binary search expects.

7. Recursive Algorithm Used

`recursiveSearchByName` walks the records one position at a time. The base case is reached when the position runs off the end of the list; the recursive case tries the next position and joins what it found. The recursion solves a genuine subproblem — the same search on the remainder of the list — rather than being a loop dressed as recursion. Merge sort is recursive in the same way: it splits the list and defers to itself on each half.

8. GUI Design

The interface is built in Qt Widgets and split into six tabs, one class per tab: Patient Records, Appointments, Waiting Queue, Search & Sort, Clinic Reports and Administration.

- **Events.** Button clicks, menu actions, combo box changes, text field changes and a one-second timer all go through signals and slots. The File menu holds New, Save as…, Load… and Exit; the Edit menu holds Undo last operation and Quick search.
- **Layouts.** No pixel positioning is used. `QFormLayout` keeps a label with its field, `QHBoxLayout` and `QVBoxLayout` divide the panels into rows and columns, and `QGridLayout` lines up the tables of figures and of category priorities. Using four techniques rather than one means each part of the window is laid out by the one that suits its shape.
- **Advanced techniques.** Rows are colour-coded by category (red for emergency, green for general, blue for follow-up) with a table of categories and priorities beside the roster; the form fields change with the chosen patient type; file dialogs choose the import and export files; the timer keeps the status bar and the dashboard live; every invalid entry raises a message naming the field; shortcuts cover Ctrl+N, Ctrl+S, Ctrl+O, Ctrl+Z, Ctrl+F and Ctrl+Q; and every control carries a tooltip.

9. Exception Handling Approach

`ClinicException`, the base of the hierarchy, derives from `std::exception` and overrides `what()`. Four derived types name the failure precisely: `DuplicateIDException` for an ID already registered, `InvalidPatientDataException` for an age outside 0–120 or a blank name, `InvalidAppointmentException` for a booking without a date or a patient that is not registered, and `FileIOException` for a file that cannot be opened, read or written.

Every panel wraps the work it asks the controller to do in `try` / `catch` and shows the message as a dialog, so a mistake never closes the program. There is also a last line of defence: the application class overrides `notify()`, the

one method every event passes through, so an exception that escapes any slot is caught and shown rather than ending the program. Rows in a data file that cannot be read are skipped and listed, and a save writes to a temporary file first, so a failed write cannot destroy the data already on disk.

10. Algorithms Chosen and Reasons

- **Two sorts, not one.** Selection sort suits the handful of records a small clinic holds, because it makes the fewest swaps and needs no scratch space; merge sort is there for a large import, because it stays at O(n log n) whatever the order of the data and preserves the order of equal records.
- **Binary search with a defensive copy.** Binary search is only correct on a sorted list, so a sorted copy is made before it is called. The panel works with the records it finds and not with their positions, so the record shown is always the record that was found.
- **One comparison rule everywhere.** The search ignores letter case, so the sort and the binary search do too, which keeps the list sorted the way the binary search expects.
- **One cache order.** The recently accessed records use a single order — oldest at the front, newest at the back — and a cache hit records the access, so the panel really shows the most recently used records.

11. Challenges Encountered

1. **One class that did too much.** The panels first called the database directly, so every rule of the clinic ended up in `ClinicDatabase`, which grew to 488 lines and mixed the data handling with the business logic — the very thing the brief forbids.
2. **The binary search showed the wrong patient.** The search returned a position, and the panel used that position in the *unsorted* roster, so the wrong patient appeared.
3. **The cache did not behave like a cache.** Three different ideas of the order were used at once, so the panel showed the records in registration order rather than use order.
4. **A failed save could destroy the data that was already saved.** The file was opened for writing straight away, so an ofstream emptied it before the writing had even begun.
5. **Loading a file threw the appointments away.** The save asked for two files but the load asked for one, so appointments saved under another name were silently lost.
6. **One mistake could end the program.** Qt does not catch an exception that leaves a slot.

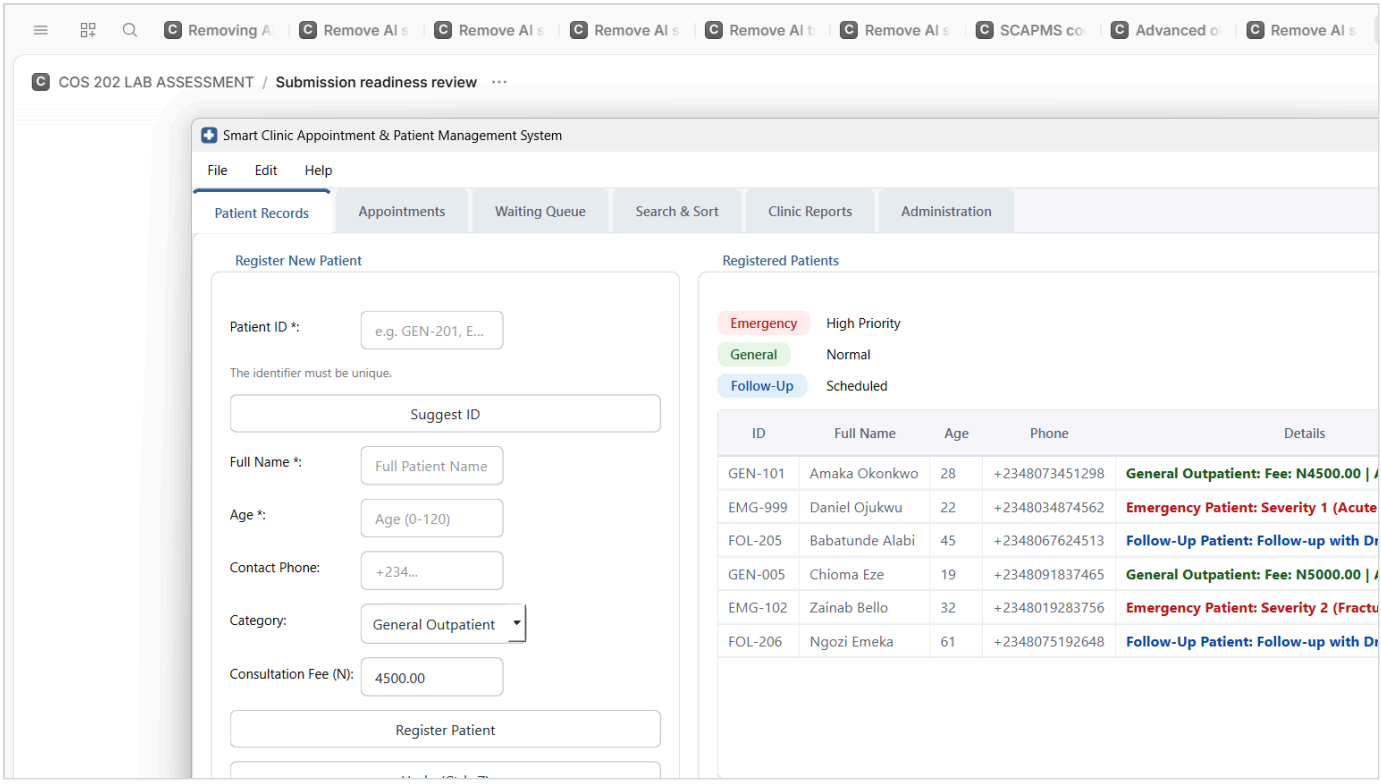
12. How the Challenges Were Resolved

1. The rules were moved up into `ClinicManager` and `AppointmentController` in the controller layer, and the panels were repointed at the manager. `ClinicDatabase` now only stores and finds, and no file in `gui/` refers to it.
2. The search returns the records that matched, not their positions, so the record shown is always the record that was found.
3. One order is used everywhere: a record used again moves to the newest place, a new record is added at the back, and a cache hit records the access.
4. The records are written to a temporary file first, and the real file is only replaced when the writing has finished, so a failed save leaves the old file untouched.
5. The load asks for the appointment file as well, offers the file next to the patient file when the staff cancels, and asks before the clinic is replaced.

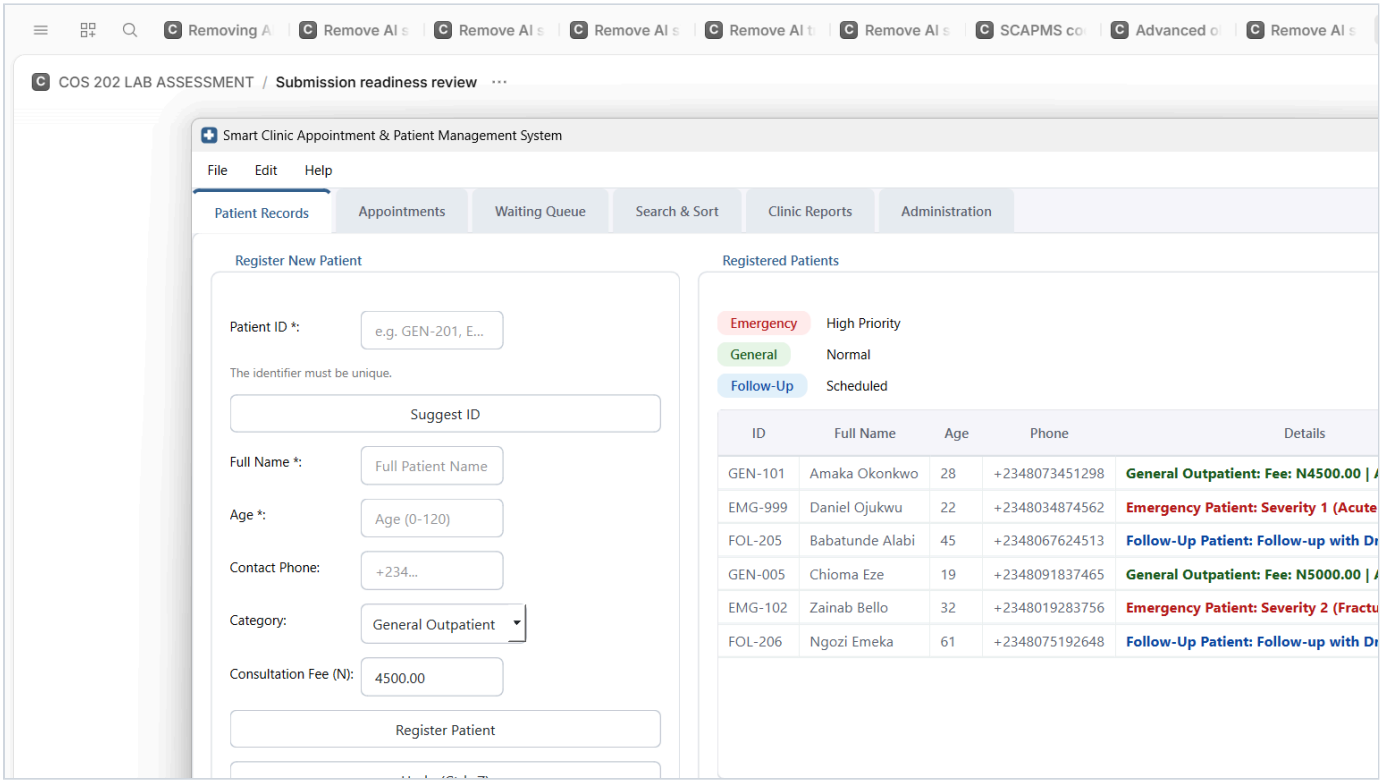
6. The application class in `gui_main.cpp` overrides `notify()`, the one place every event passes through, so anything that escapes a slot is caught and shown as a message.

The complete source code is at `https://github.com/OlisaDavies/COS-202-LAB-ASSESSMENT` and is also listed in the Source Code section of this document.

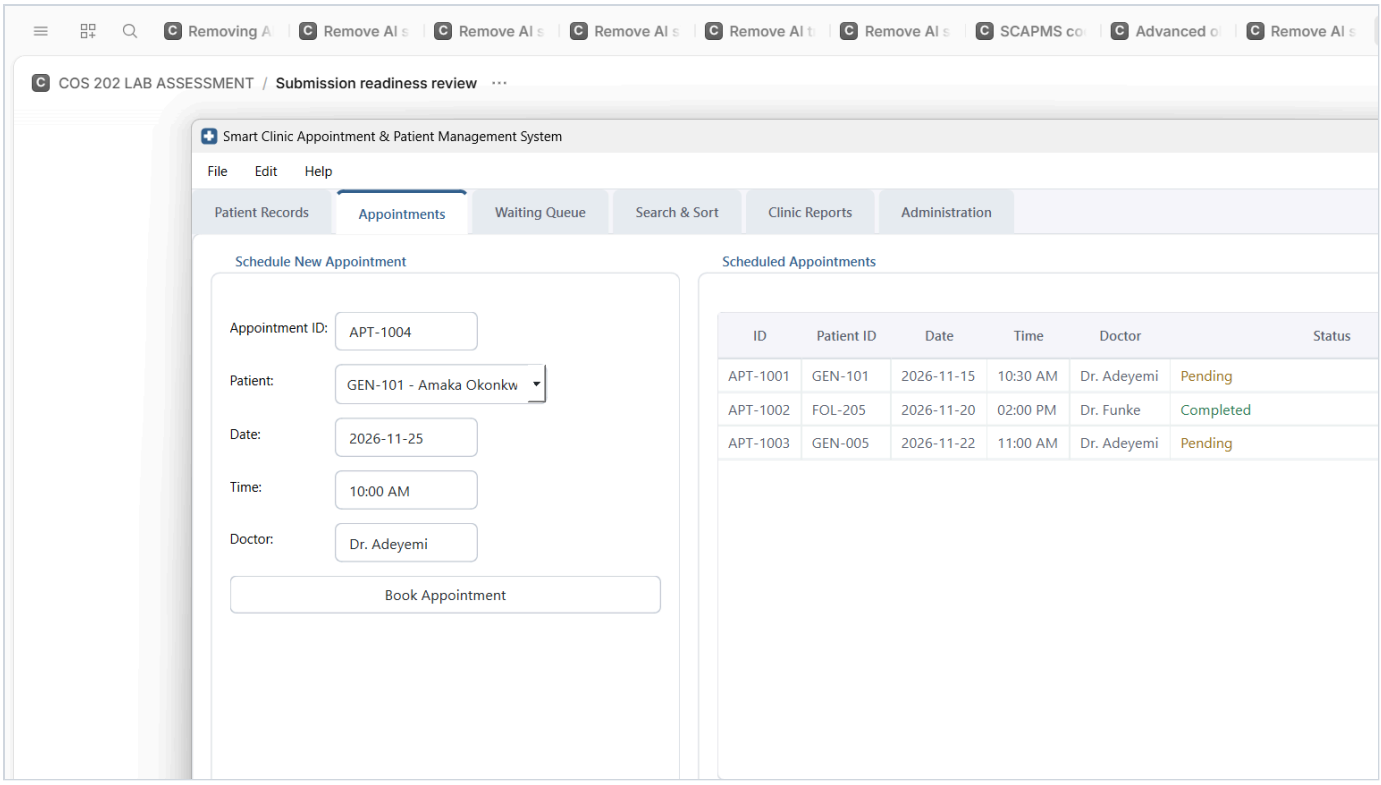
2. Screenshots



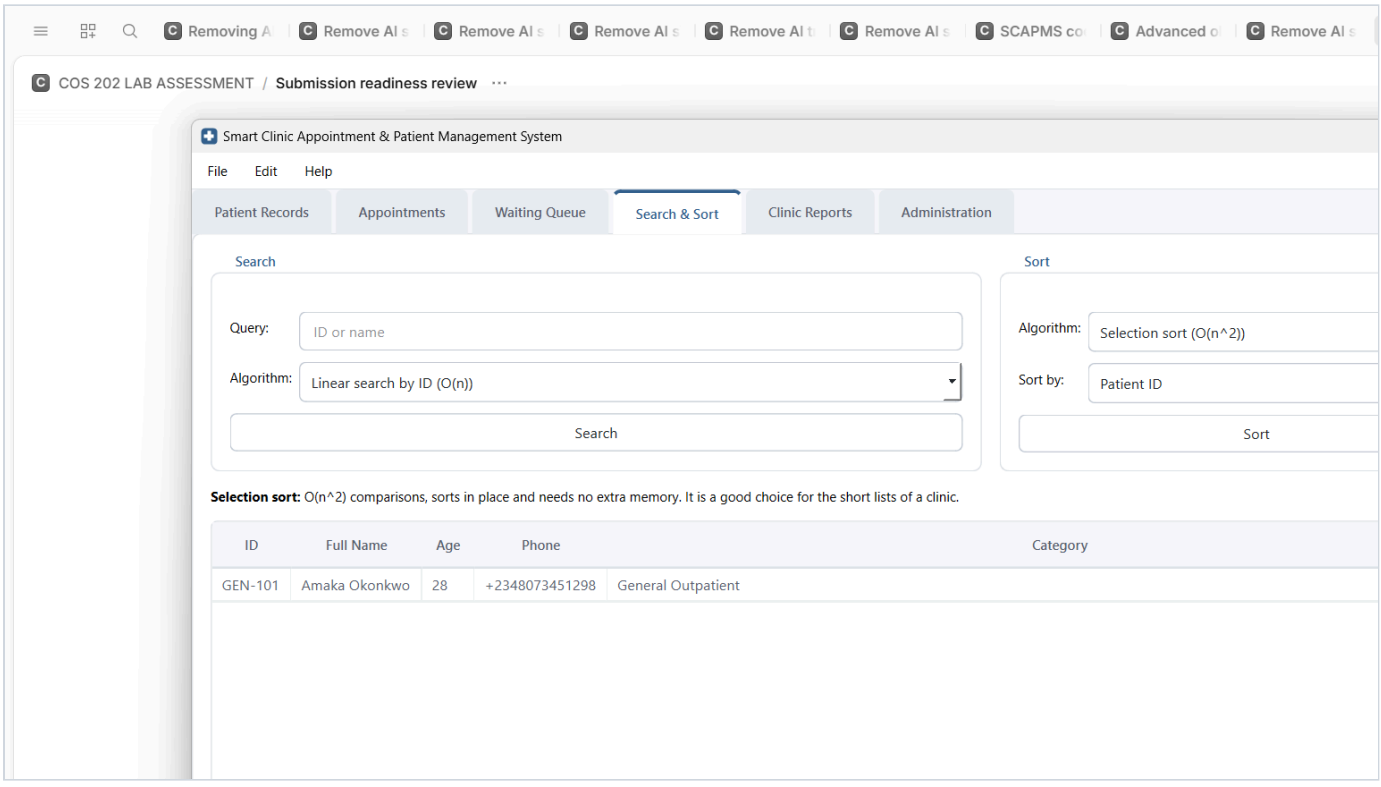
1_main_dashboard.png — Main dashboard - the live figures of the clinic, the category priorities and the recently accessed records



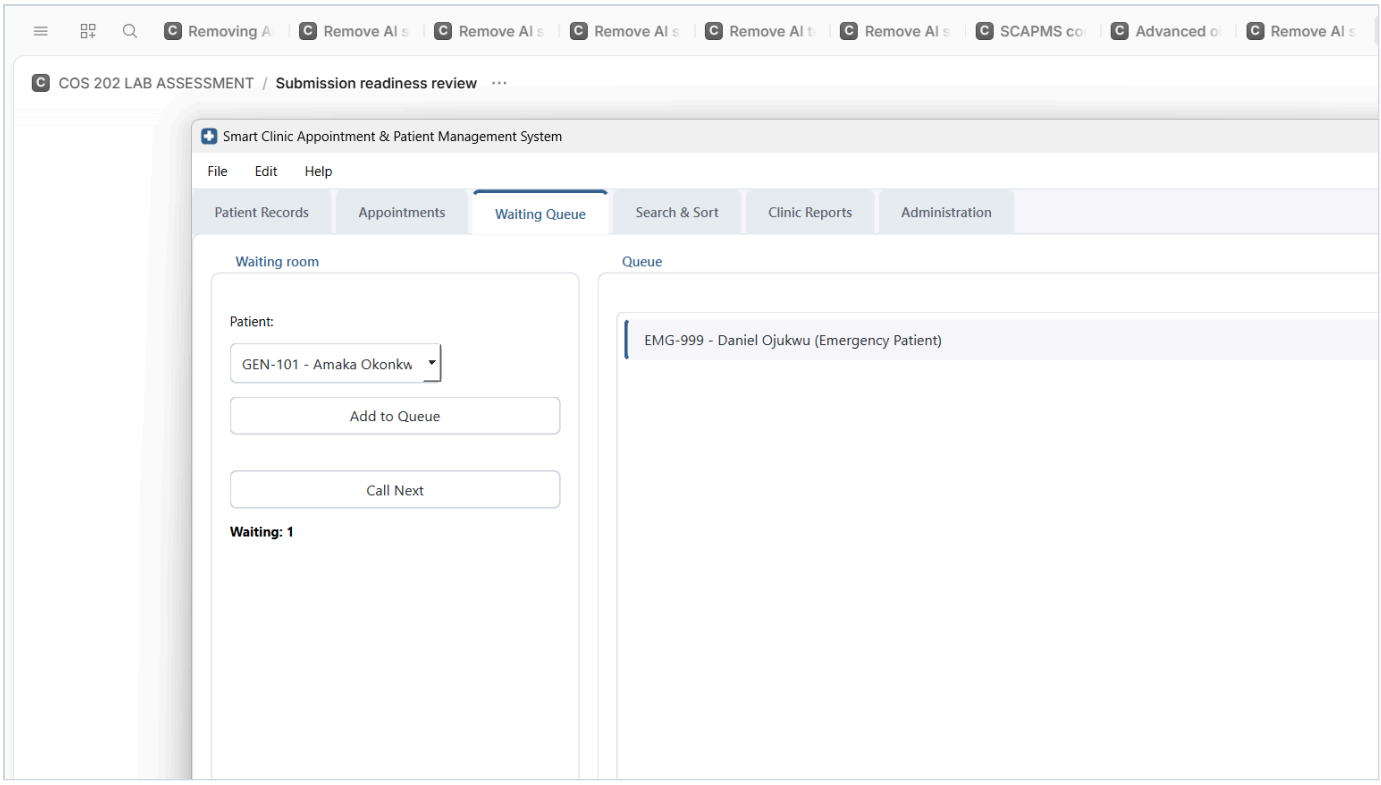
2_patient_registration.png — Patient registration - the registration form and the colour-coded roster of the patients



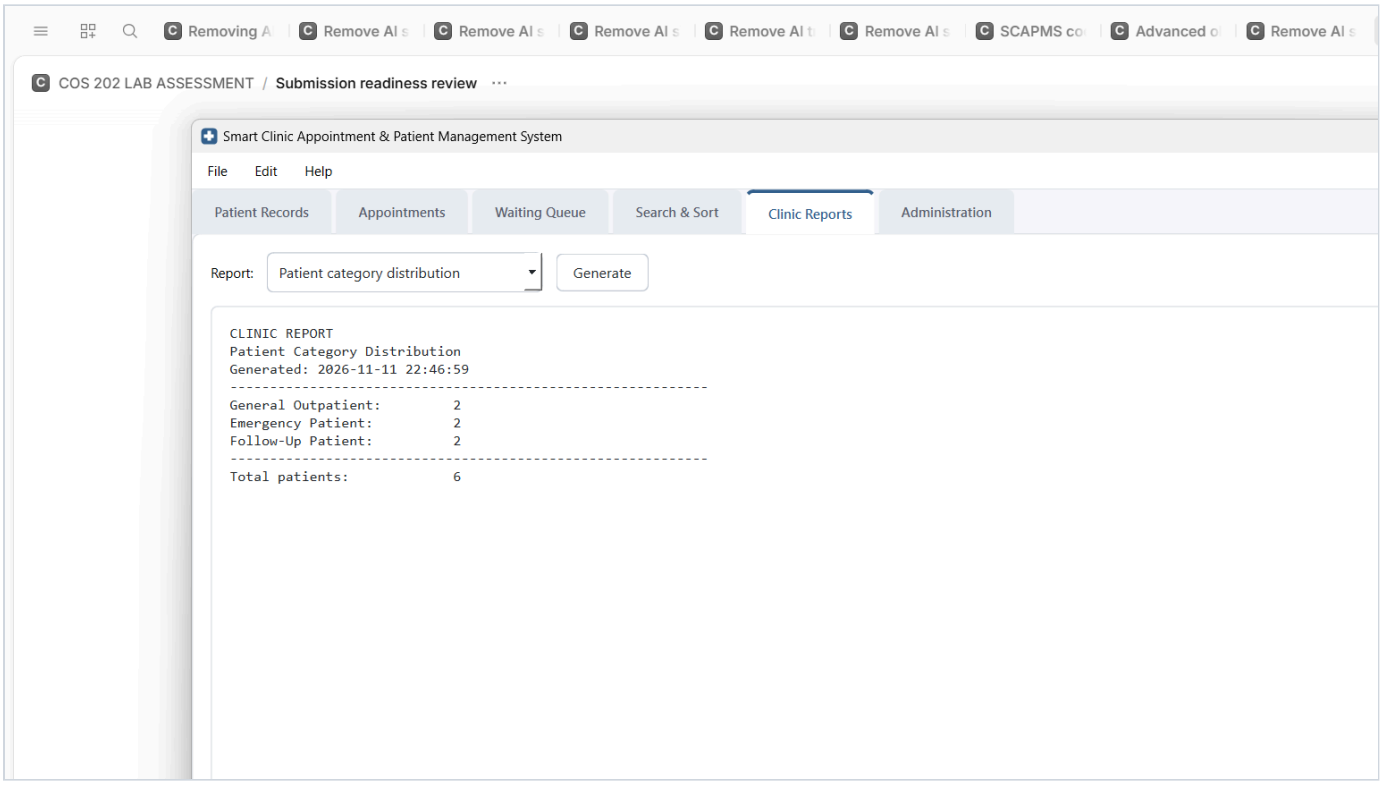
3_appointment_management.png — Appointment management - the booking form and the schedule of the clinic



4_search_and_sort.png — Search and sort - the algorithm choices and the result of a linear search by patient ID



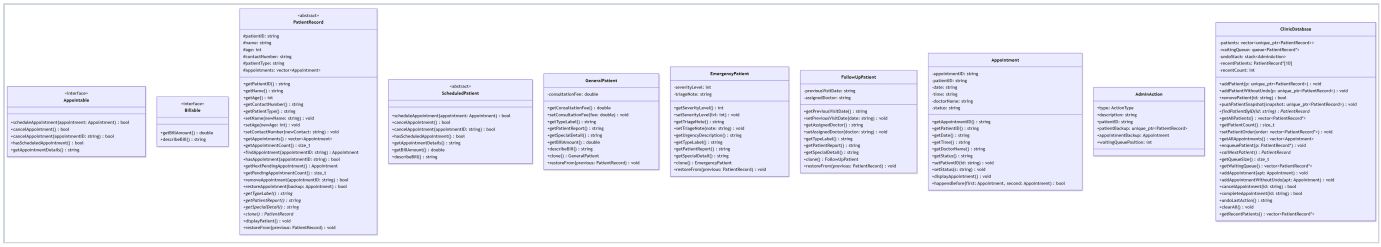
5_waiting_queue.png — Waiting queue - a patient in the consultation queue



6_clinic_reports.png — Clinic reports - the patient category distribution

3. UML Class Diagrams

The full diagram of the program covers twenty five classes and is too tall to read on one page, so it is drawn as the four layers of the program. Each page holds the classes of its own layer, the relationships between them, and the single relationship that joins the layer to the next one, so every arrow of the full diagram is still present. The inheritance arrows, the `«abstract»` and `«interface»` stereotypes, the composition markers and the virtual calls in the base classes show the four things the brief asks a diagram to demonstrate.



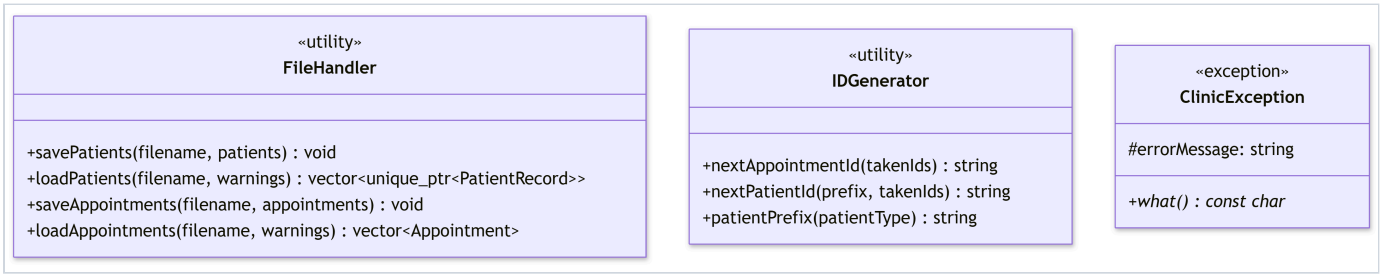
UML_1_model.png — Model layer - the abstract base class, the three patient categories, the appointment and the database



UML_2_controller.png — Controller layer - the manager, the appointment rules and the two algorithm engines



UML_3_screens.png — Interface layer - the main window and one class per tab



UML_4_utilities.png — Utility layer - the file handler, the identifier generator and the exception classes

4. Source Code

The complete source of the program, grouped under the four headings the brief names. The model, controller and utility layers are plain C++ and use no Qt at all, so the same code runs in the console demonstration (`main.cpp`) and in the graphical program (`gui_main.cpp`).

4.1 Header files

controller/AppointmentController.h

```
#pragma once

#include <string>
#include <vector>

#include "../model/ClinicDatabase.h"
#include "../model/Appointment.h"

namespace scapms {
namespace controller {

// The rules that govern an appointment.
//
// The controller checks the rules of the clinic and then hands the booking to
// the database, which only stores it. That keeps the two apart: the rules are
// here, the data is in the model layer.
class AppointmentController {
public:
    explicit AppointmentController(model::ClinicDatabase& database);

    // Books an appointment for a patient.
    //
    // The date and the time have to be written in the format the clinic
    // uses, the patient has to be registered, the doctor must be free in
    // that slot and the category of the patient has to be one that can be
    // booked in advance. Every rule that is broken is reported with an
    // InvalidAppointmentException.
    //
    // A booking that comes from a file is not an administrative step of its
    // own, so it is not remembered on the undo stack.
    void scheduleAppointment(const model::Appointment& appointment, bool trackUndo = true);

    // Cancels the appointment with the given identifier. Returns false when
    // no such appointment is booked.
    bool cancelAppointment(const std::string& appointmentID);

    // Marks a visit as completed, so that it counts towards the attendance
    // of the patient. Returns false when the appointment is not booked, or
    // when it was cancelled already.
    bool completeAppointment(const std::string& appointmentID);

    std::string suggestAppointmentId() const;

private:
    void checkBookingRules(const model::Appointment& appointment) const;

    // True when the patient is registered and can be booked in advance.
    bool patientCanBeBooked(const model::Appointment& appointment) const;

    model::ClinicDatabase& database;
};

} // namespace controller
} // namespace scapms
```

controller/ClinicManager.h

```
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "../model/ClinicDatabase.h"
#include "../model/PatientRecord.h"
#include "../model/Appointment.h"
#include "AppointmentController.h"

namespace scapms {
namespace controller {

// The controller of the clinic, and the only class the graphical interface
// talks to. The screens never touch the database directly, and the database
// never decides what is allowed, so every rule of the clinic is written down
// in this one place.
class ClinicManager {
public:
    ClinicManager();
    ~ClinicManager();

    // The manager owns its data, so it cannot be copied.
    ClinicManager(const ClinicManager&) = delete;
    ClinicManager& operator=(const ClinicManager&) = delete;

    // Registers a new patient. The record is checked against the rules of
    // the clinic first, so a record that breaks a rule is refused with an
    // exception and the caller is told why. The step is remembered, so it
    // can be undone.
    void registerPatient(std::unique_ptr<model::PatientRecord> patient);

    // Registers several patients at once, for example after a file was read.
    // The records that break a rule are skipped and described in the
    // warnings list. The load is one administrative step as a whole, so the
    // records are not remembered one by one.
    void registerPatients(std::vector<std::unique_ptr<model::PatientRecord>> patients,
                          std::vector<std::string>* warnings = nullptr);

    // Checks a record against the rules of the clinic without storing it.
    // The edit panel uses this, so that an edit cannot put an illegal
    // value into a record that was registered properly.
    void validatePatient(const model::PatientRecord& patient) const {
        checkPatientRules(patient);
    }

    // Deletes a patient. The step can be undone.
    bool deletePatient(const std::string& patientID);

    model::PatientRecord* findPatientByID(const std::string& patientID);
    const model::PatientRecord* findPatientByID(const std::string& patientID) const;

    // Every registered patient. The records belong to the manager.
    std::vector<model::PatientRecord*> getAllPatients() const;
    std::size_t getPatientCount() const;

    // Puts the records in the given order, so that the roster and the
    // search results show the same order.
    void setPatientOrder(const std::vector<model::PatientRecord*>& order);

    // The next free identifier for a category, for example "GEN-106".
    std::string suggestPatientId(const std::string& patientType) const;

    // Keeps a copy of a record so that the change can be undone later.
    void pushPatientSnapshot(std::unique_ptr<model::PatientRecord> snapshot);

    // Books an appointment. The rules of the clinic are checked first.
    // A booking that comes from a file is not remembered on the undo stack.
```

controller/ReportBuilder.h

```
#pragma once

#include <string>
#include <vector>

#include "ClinicManager.h"

namespace scapms {
namespace controller {

// Builds the clinic reports.
//
// The reports are plain text so that they can be printed by the console
// demonstration and shown in the GUI without the report code depending on the
// user interface.
class ReportBuilder {
public:
    static std::string categoryDistribution(const ClinicManager& manager);

    // Every appointment that has not been completed yet.
    static std::string pendingAppointments(const ClinicManager& manager);

    // How many appointments were scheduled, completed, cancelled or are still
    // pending.
    static std::string dailySummary(const ClinicManager& manager);

    // The patients with the most completed visits, best first.
    static std::string frequentPatients(const ClinicManager& manager);

    // The short version shown on the dashboard.
    static std::string dashboardSummary(const ClinicManager& manager);

private:
    static std::string header(const std::string& title);

    static std::string timestamp();

    static std::vector<std::pair<std::string, int>> completedVisitCounts(
        const ClinicManager& manager);
};

} // namespace controller
} // namespace scapms
```

controller/SearchEngine.h

```
#pragma once

#include <string>
#include <vector>

#include "../model/PatientRecord.h"

namespace scapms {
namespace controller {

// The searching routines of the clinic. Every one of them is written by hand,
// so the program never calls the standard library search functions.
//
// Which routine to use depends on the state of the data:
//   * linear search works on a list in any order,
//   * binary search needs a list that is already sorted,
//   * the recursive search walks the list step by step.
class SearchEngine {
public:
    // Scans the whole list and returns the position of the patient with the
    // given ID, or -1. Works on a list that has never been sorted.
    static int linearSearchByID(const std::vector<model::PatientRecord*>& records,
                                const std::string& targetID);

    // Scans the whole list and returns the position of every patient whose
    // identifier or whose name holds the given text.
    //
    // The text is matched without regard to letter case, and a part of the
    // identifier or of the name is enough. That is what the clinic staff
    // expect, because they type "chi" and mean "Chioma".
    static std::vector<int> linearSearchByText(const std::vector<model::PatientRecord*>& records,
                                               const std::string& text);

    // Halves the search space on every step. The list must be sorted by
    // patient ID, otherwise the result is meaningless.
    static int binarySearchByID(const std::vector<model::PatientRecord*>& sortedRecords,
                                const std::string& targetID);

    // Looks for patients by name. The search calls itself with the next
    // position until it runs off the end of the list, and it collects every
    // patient whose name holds the given text. The text is matched without
    // regard to letter case.
    static std::vector<int> recursiveSearchByName(const std::vector<model::PatientRecord*>& records,
                                                  const std::string& targetName,
                                                  int currentIndex = 0);

    static std::vector<int> searchByCategory(const std::vector<model::PatientRecord*>& records,
                                             const std::string& category);
};

} // namespace controller
} // namespace scapms
```

controller/SortEngine.h

```
#pragma once

#include <string>
#include <vector>

#include "../model/PatientRecord.h"

namespace scapms {
namespace controller {

// The field the records are ordered by.
enum class SortField {
    PatientID,
    PatientName,
    PatientAge
};

// The sorting routines of the clinic. Both are written by hand.
//
// Selection sort is the simple quadratic routine, merge sort is the divide
// and conquer routine that always needs O(n log n) comparisons.
class SortEngine {
public:
    static bool comparePatients(const model::PatientRecord* first,
                                const model::PatientRecord* second,
                                SortField field);

    // Orders the records with selection sort. It sorts in place and performs
    // at most n - 1 swaps, which suits small lists.
    static void selectionSort(std::vector<model::PatientRecord*>& records, SortField field);

    // Orders the records with merge sort. It is stable, so records with the
    // same key keep the order they had before.
    static void mergeSort(std::vector<model::PatientRecord*>& records, SortField field);

private:
    // Splits the range in two and sorts both halves.
    static void mergeSortRecursive(std::vector<model::PatientRecord*>& records,
                                   int left, int right, SortField field);

    // Joins two sorted halves back into one sorted range.
    static void mergeHalves(std::vector<model::PatientRecord*>& records,
                            int left, int middle, int right, SortField field);
};

} // namespace controller
} // namespace scapms
```

model/AdminAction.h

```
#pragma once

#include <memory>
#include <string>

#include "PatientRecord.h"
#include "Appointment.h"

namespace scapms {
namespace model {

// One entry on the administrative undo stack.
//
// An entry remembers what was done, which record it was about and a copy of
// that record. The copy is a separate object, so the undo stack and the
// roster never share the same memory.
//
// The class owns a record, so it follows the rule of five: it can be moved
// but not copied.
class AdminAction {
public:
    enum class Type {
        AddPatient,
        DeletePatient,
        UpdatePatient,
        AddAppointment,
        CancelAppointment,
        CompleteAppointment
    };

    Type type;
    std::string description;
    std::string patientID;
    std::unique_ptr<PatientRecord> patientBackup;
    Appointment appointmentBackup;

    // The place the record held in the waiting queue when it was deleted,
    // counted from the front. A value of -1 means that the record was not
    // waiting, so undoing the delete must not put it back into the queue.
    int waitingQueuePosition;

    AdminAction(Type actionType, const std::string& text, const std::string& affectedPatientID);
    AdminAction(Type actionType, const std::string& text, const std::string& affectedPatientID,
                std::unique_ptr<PatientRecord> backup);
    AdminAction(Type actionType, const std::string& text, const std::string& affectedPatientID,
                const Appointment& appointment);

    // The entry owns a record, so copying it would copy the ownership as
    // well. It can be moved, which is what the stack does.
    AdminAction(const AdminAction&) = delete;
    AdminAction& operator=(const AdminAction&) = delete;
    AdminAction(AdminAction&&) = default;
    AdminAction& operator=(AdminAction&&) = default;
    ~AdminAction() = default;
};

} // namespace model
} // namespace scapms
```

model/Appointable.h

```
#pragma once

#include <string>

#include "Appointment.h"

namespace scapms {
namespace model {

// Interface for the patient types that can be booked for appointments. It
// only declares operations and holds no data, so any class that can be
// scheduled can implement it. Emergency patients deliberately do not, because
// a trauma case is never booked in advance.
class Appointable {
public:
    virtual ~Appointable() = default;

    // Books the given appointment for this patient. The patient keeps the
    // appointment in its own collection, so the booking and the patient are
    // always in step. Returns false when the booking is not allowed.
    virtual bool scheduleAppointment(const Appointment& appointment) = 0;

    // Cancels the booking that comes first. Returns false when the patient
    // holds no booking at all.
    virtual bool cancelAppointment() = 0;

    // Cancels the appointment with the given identifier. Returns false when
    // the patient does not hold that appointment.
    virtual bool cancelAppointment(const std::string& appointmentID) = 0;

    // True when the patient currently holds a booking that has not been
    // completed or cancelled yet.
    virtual bool hasScheduledAppointment() const = 0;

    // A short description of the booking that is due next.
    virtual std::string getAppointmentDetails() const = 0;
};

} // namespace model
} // namespace scapms
```

model/Appointment.h

```
#pragma once

#include <string>

namespace scapms {
namespace model {

// One booked consultation.
//
// The appointment belongs to a patient. The patient holds the appointment in
// its own collection, so a patient can have several appointments at the
// same time. The identifier of the patient is kept in the appointment as
// well, because the flat schedule that is written to the data file needs it.
class Appointment {
public:
    // The three states an appointment can be in. They are kept here so that
    // the same word is used everywhere in the program.
    static constexpr const char* STATUS_PENDING   = "Pending";
    static constexpr const char* STATUS_COMPLETED = "Completed";
    static constexpr const char* STATUS_CANCELLED = "Cancelled";

    // True when the first appointment happens before the second one. The
    // times are compared as minutes since midnight, so that "09:00 AM" is
    // correctly placed before "10:30 AM".
    static bool happensBefore(const Appointment& first, const Appointment& second);

private:
    std::string appointmentID;
    std::string patientID;
    std::string date;        // YYYY-MM-DD
    std::string time;        // HH:MM AM / PM
    std::string doctorName;
    std::string status;

public:
    Appointment(std::string aptId, std::string patId, std::string aptDate,
                std::string aptTime, std::string docName,
                std::string aptStatus = STATUS_PENDING)
        : appointmentID(aptId),
          patientID(patId),
          date(aptDate),
          time(aptTime),
          doctorName(docName),
          status(aptStatus) {}

    const std::string& getAppointmentID() const { return appointmentID; }
    const std::string& getPatientID() const { return patientID; }
    const std::string& getDate() const { return date; }
    const std::string& getTime() const { return time; }
    const std::string& getDoctorName() const { return doctorName; }
    const std::string& getStatus() const { return status; }

    // The patient that owns the appointment. It is set by the patient itself
    // when the appointment is stored, so that the two always agree.
    void setPatientID(std::string id) { patientID = id; }

    void setDate(std::string newDate) { date = newDate; }
    void setTime(std::string newTime) { time = newTime; }
    void setDoctorName(std::string newDoctor) { doctorName = newDoctor; }
    void setStatus(std::string newStatus) { status = newStatus; }

    void displayAppointment() const;
};

} // namespace model
} // namespace scapms
```

model/Billable.h

```
#pragma once

#include <string>

namespace scapms {
namespace model {

// Interface for the patient types that the clinic can send a bill to.
//
// Like Appointable it is a pure abstract class: it declares the operations
// and holds no member variables. A class may implement more than one
// interface, and ScheduledPatient implements both of them.
class Billable {
public:
    virtual ~Billable() = default;

    // The amount that has to be paid for the consultation, in naira.
    virtual double getBillAmount() const = 0;

    // A short description of the bill, for the screen and for the reports.
    virtual std::string describeBill() const = 0;
};

} // namespace model
} // namespace scapms
```

model/ClinicDatabase.h

```
#pragma once

#include <memory>
#include <queue>
#include <stack>
#include <string>
#include <vector>

#include "PatientRecord.h"
#include "Appointment.h"
#include "Appointable.h"
#include "AdminAction.h"

namespace scapms {
namespace model {

// The store of the clinic: the patient records, the waiting room and the
// undo history. It only stores and finds; the rules of the clinic live in
// the controller layer. The database owns the records and deletes them, the
// undo stack owns one copy per action, and the waiting room only borrows.
class ClinicDatabase {
public:
    static constexpr int CACHE_SIZE = 10;

private:
    // The database owns every patient record. The unique pointers make sure
    // that a record is freed exactly once, even when the database is
    // cleared or an action is undone.
    std::vector<std::unique_ptr<PatientRecord>> patients;
    std::queue<PatientRecord*> waitingQueue;
    std::stack<AdminAction> undoStack;

    // Fixed size cache of the most recently accessed records.
    //
    // The array is held from the oldest record at the front to the newest
    // one at the back, and only the first recentCount places of it are in
    // use. When it is full, the record at the front is the one that was
    // used the longest time ago, and it makes room for a new record.
    PatientRecord* recentPatients[CACHE_SIZE];
    int recentCount;

    // Adds one patient, with or without an entry on the undo stack.
    void addPatientInternal(std::unique_ptr<PatientRecord> patient, bool trackUndo);

    // Adds one appointment to the patient that owns it, with or without an
    // entry on the undo stack.
    void addAppointmentInternal(const Appointment& appointment, bool trackUndo);

    // Removes a record without touching the undo history. When backupOut is
    // given it receives a copy of the removed record. When waitingPositionOut
    // is given it receives the place the record held in the waiting queue,
    // or -1 when it was not waiting.
    bool removePatientInternal(const std::string& id,
                               std::unique_ptr<PatientRecord>* backupOut,
                               int* waitingPositionOut);

    PatientRecord* findOwnerOfAppointment(const std::string& appointmentID);

    bool appointmentIDIsTaken(const std::string& appointmentID) const;

    void touchRecentPatient(PatientRecord* patient);
    void removeRecentPatient(PatientRecord* patient);
    void removeFromWaitingQueue(PatientRecord* patient);

    // The place the record holds in the waiting queue, counted from the
    // front, or -1 when it is not waiting.
    int waitingQueuePosition(const PatientRecord* patient) const;

    // Puts a record back into the waiting queue at the given place. A
```

model/EmergencyPatient.h

```
#pragma once

#include <string>

#include "PatientRecord.h"

namespace scapms {
namespace model {

// An urgent case that is treated as soon as it arrives.
//
// It deliberately does not implement Appointable: a trauma case is never
// booked in advance, it goes straight into the waiting room.
class EmergencyPatient : public PatientRecord {
private:
    int severityLevel;      // 1 = Critical, 2 = Urgent, 3 = Moderate
    std::string triageNote;

public:
    EmergencyPatient(std::string id, std::string patientName, int patientAge,
                     std::string contact, int severity, std::string note)
        : PatientRecord(id, patientName, patientAge,
                        contact, PatientRecord::TYPE_EMERGENCY),
          severityLevel(severity),
          triageNote(note) {}

    EmergencyPatient(const EmergencyPatient& other) = default;

    int getSeverityLevel() const { return severityLevel; }
    void setSeverityLevel(int level) { severityLevel = level; }

    const std::string& getTriageNote() const { return triageNote; }
    void setTriageNote(std::string note) { triageNote = note; }

    std::string getUrgencyDescription() const;

    std::string getTypeLabel() const override { return "Emergency Patient"; }

    std::string getPatientReport() const override;

    std::string getSpecialDetail() const override;

    // The covariant return type hands back an EmergencyPatient.
    EmergencyPatient* clone() const override { return new EmergencyPatient(*this); }

    void restoreFrom(const PatientRecord& previous) override;
};

} // namespace model
} // namespace scapms
```

model/FollowUpPatient.h

```
#pragma once

#include <string>

#include "ScheduledPatient.h"

namespace scapms {
namespace model {

// A returning patient that is seen again for the same complaint. The record
// keeps the previous visit and the doctor who is looking after the case.
class FollowUpPatient : public ScheduledPatient {
private:
    std::string previousVisitDate;
    std::string assignedDoctor;

public:
    FollowUpPatient(std::string id, std::string patientName, int patientAge,
                    std::string contact, std::string prevDate, std::string doctor)
        : ScheduledPatient(id, patientName, patientAge,
                           contact, PatientRecord::TYPE_FOLLOWUP),
          previousVisitDate(prevDate),
          assignedDoctor(doctor) {}

    FollowUpPatient(const FollowUpPatient& other) = default;

    const std::string& getPreviousVisitDate() const { return previousVisitDate; }
    void setPreviousVisitDate(std::string date) { previousVisitDate = date; }

    const std::string& getAssignedDoctor() const { return assignedDoctor; }
    void setAssignedDoctor(std::string doctor) { assignedDoctor = doctor; }

    std::string getTypeLabel() const override { return "Follow-Up Patient"; }

    std::string getPatientReport() const override;

    std::string getSpecialDetail() const override;

    // The covariant return type hands back a FollowUpPatient.
    FollowUpPatient* clone() const override { return new FollowUpPatient(*this); }

    void restoreFrom(const PatientRecord& previous) override;
};

} // namespace model
} // namespace scapms
```

model/GeneralPatient.h

```
#pragma once

#include <cmath>
#include <string>

#include "ScheduledPatient.h"

namespace scapms {
namespace model {

// A standard outpatient visit. The patient pays a consultation fee and can be
// booked for a scheduled consultation.
class GeneralPatient : public ScheduledPatient {
private:
    double consultationFee;

public:
    // The consultation fee of the clinic. The rule lives with the data, so
    // that the screen, the controller and the file reader all use the same
    // numbers.
    static constexpr double MAX_CONSULTATION_FEE = 100000000.0;
    static constexpr double MIN_CONSULTATION_FEE = 0.0;

    GeneralPatient(std::string id, std::string patientName, int patientAge,
                   std::string contact, double fee)
        : ScheduledPatient(id, patientName, patientAge,
                           contact, PatientRecord::TYPE_GENERAL),
          consultationFee(fee) {}

    GeneralPatient(const GeneralPatient& other) = default;

    double getConsultationFee() const { return consultationFee; }
    void setConsultationFee(double fee) { consultationFee = fee; }

    // True when the fee is a real amount that the clinic can bill. A value
    // of "nan" or of infinity is not a real amount, even though every
    // comparison with it is false.
    static bool isValidConsultationFee(double fee) {
        return std::isfinite(fee) &&
               fee >= MIN_CONSULTATION_FEE &&
               fee <= MAX_CONSULTATION_FEE;
    }

    std::string getTypeLabel() const override { return "General Outpatient"; }

    std::string getPatientReport() const override;

    std::string getSpecialDetail() const override;

    // A general outpatient visit is billed for the consultation fee.
    double getBillAmount() const override;
    std::string describeBill() const override;

    // The covariant return type hands back a GeneralPatient, so the caller
    // does not have to cast the copy back down.
    GeneralPatient* clone() const override { return new GeneralPatient(*this); }

    void restoreFrom(const PatientRecord& previous) override;
};

} // namespace model
} // namespace scapms
```

model/PatientRecord.h

```
#pragma once

#include <string>
#include <vector>

#include "Appointment.h"

namespace scapms {
namespace model {

// Abstract base class for every patient the clinic knows about. It holds the
// details every category has in common and leaves the rest to the subclasses.
// It declares pure virtual functions, so it can never be created on its own,
// and it also carries concrete methods that every category shares.
class PatientRecord {
public:
    // The three patient categories. The strings are also the key used in the
    // data files, so they are kept in one place.
    static constexpr const char* TYPE_GENERAL   = "General";
    static constexpr const char* TYPE_EMERGENCY = "Emergency";
    static constexpr const char* TYPE_FOLLOWUP  = "FollowUp";

protected:
    std::string patientID;
    std::string name;
    int age;
    std::string contactNumber;
    std::string patientType;

    // Composition: a patient holds the appointments that belong to it, so a
    // patient is associated with one or more appointments. The appointments
    // are stored by value, so they are created and destroyed together with
    // the patient that owns them.
    std::vector<Appointment> appointments;

public:
    PatientRecord(std::string id, std::string patientName, int patientAge,
                  std::string contact, std::string type)
        : patientID(id),
          name(patientName),
          age(patientAge),
          contactNumber(contact),
          patientType(type) {}

    // Copying is allowed so that a record can be cloned safely for the undo
    // stack and for the edit snapshot. The copy is deep: the appointment
    // collection is copied as well.
    PatientRecord(const PatientRecord& other) = default;
    PatientRecord& operator=(const PatientRecord& other) = default;

    // Virtual destructor, so that deleting a record through a PatientRecord*
    // also runs the destructor of the subclass.
    virtual ~PatientRecord() = default;

    const std::string& getPatientID() const { return patientID; }
    const std::string& getName() const { return name; }
    int getAge() const { return age; }
    const std::string& getContactNumber() const { return contactNumber; }
    const std::string& getPatientType() const { return patientType; }

    void setName(std::string newName) { name = newName; }
    void setAge(int newAge) { age = newAge; }
    void setContactNumber(std::string newContact) { contactNumber = newContact; }

    // The appointments the patient holds. This is the association that the
    // UML diagram shows as a composition.
    const std::vector<Appointment>& getAppointments() const { return appointments; }

    std::size_t getAppointmentCount() const { return appointments.size(); }
```

model/ScheduledPatient.h

```
#pragma once

#include <string>

#include "PatientRecord.h"
#include "Appointable.h"
#include "Billable.h"

namespace scapms {
namespace model {

// Intermediate abstract class for the patient types that can be booked. It
// inherits from the base class and from two interfaces at once, so a class
// may implement more than one contract, and it writes the booking code once
// for GeneralPatient and FollowUpPatient.
class ScheduledPatient : public PatientRecord, public Appointable, public Billable {
public:
    ScheduledPatient(std::string id, std::string patientName, int patientAge,
                     std::string contact, std::string type)
        : PatientRecord(id, patientName, patientAge, contact, type) {}

    ScheduledPatient(const ScheduledPatient& other) = default;

    // Books the appointment in this patient's own collection. The booking is
    // refused when it has no date or no time.
    bool scheduleAppointment(const Appointment& appointment) override;

    bool cancelAppointment() override;

    // Marks the appointment with that identifier as cancelled. The booking
    // stays in the collection, because the clinic keeps a record of it.
    bool cancelAppointment(const std::string& appointmentID) override;

    bool hasScheduledAppointment() const override;

    std::string getAppointmentDetails() const override;

    // The billing contract is answered here with the default of a category
    // that does not charge for its visits. The categories that do charge
    // override the two methods.
    double getBillAmount() const override;
    std::string describeBill() const override;
};

} // namespace model
} // namespace scapms
```

utils/ClinicExceptions.h

```
#pragma once

#include <exception>
#include <string>

namespace scapms {
namespace utils {

// Base class for every error the clinic system raises on purpose.
// The GUI catches this one type and turns it into a message box, so a new
// error type never needs a new catch block.
class ClinicException : public std::exception {
protected:
    std::string errorMessage;

public:
    explicit ClinicException(const std::string& message) : errorMessage(message) {}
    const char* what() const noexcept override { return errorMessage.c_str(); }
};

class DuplicateIDException : public ClinicException {
public:
    explicit DuplicateIDException(const std::string& id)
        : ClinicException("Patient ID [" + id + "] is already registered.") {}
};

// Raised for illegal patient details such as a bad age or a blank name.
class InvalidPatientDataException : public ClinicException {
public:
    explicit InvalidPatientDataException(const std::string& detail) : ClinicException(detail) {}
};

// Raised for illegal appointment details such as a missing date or a clash.
class InvalidAppointmentException : public ClinicException {
public:
    explicit InvalidAppointmentException(const std::string& reason) : ClinicException(reason) {}
};

// Raised when a data file cannot be opened, read or written.
class FileIOException : public ClinicException {
public:
    FileIOException(const std::string& filename, const std::string& reason)
        : ClinicException("Cannot use the file [" + filename + "]: " + reason +
                          ". Please check that the path is right and that the file is not "
                          "open in another program.") {}
};

} // namespace utils
} // namespace scapms
```

utils/DateUtil.h

```
#pragma once

#include <string>

namespace scapms {
namespace utils {

// Turns a clinic time such as "02:00 PM" into the number of minutes since
// midnight, so that times can be compared instead of sorted as text.
// Returns -1 when the text is not a time.
int timeToMinutes(const std::string& time);

// True when the text looks like a calendar date, "YYYY-MM-DD".
bool isValidDate(const std::string& date);

} // namespace utils
} // namespace scapms
```

utils/FileHandler.h

```
#pragma once

#include <iosfwd>
#include <memory>
#include <string>
#include <vector>

#include "ClinicExceptions.h"
#include "../model/PatientRecord.h"
#include "../model/Appointment.h"

namespace scapms {
namespace utils {

// Reads and writes the clinic data files.
//
// Patients are stored as "type,id,name,age,contact,<category fields>" and
// appointments as "id,patient,date,time,doctor,status". Fields that contain a
// comma are written inside double quotes.
class FileHandler {
public:
    // Writes the patients. The caller keeps ownership of the records.
    static void savePatients(const std::string& filename,
                             const std::vector<model::PatientRecord*>& patients);

    static void saveAppointments(const std::string& filename,
                                 const std::vector<model::Appointment>& appointments);

    // Reads the patients. The returned records are owned by the caller, so a
    // file that is only half read never leaks them. Rows that cannot be read
    // are skipped and described in the warnings list.
    static std::vector<std::unique_ptr<model::PatientRecord>> loadPatients(
        const std::string& filename, std::vector<std::string>* warnings = nullptr);

    // Reads the appointments. Rows that cannot be read are skipped and
    // described in the warnings list.
    static std::vector<model::Appointment> loadAppointments(const std::string& filename,
                                                            std::vector<std::string>* warnings = nullptr);

private:
    // Splits one record into its fields. A field inside double quotes may
    // contain a comma, and a doubled quote is read as one quote character.
    static std::vector<std::string> splitRow(const std::string& line);

    // Wraps a field in double quotes when it contains a comma or a quote, so
    // that reading the file back gives the original value.
    static std::string quoteField(const std::string& value);

    static void writeCategoryFields(std::ostream& out, const model::PatientRecord* patient);

    static std::unique_ptr<model::PatientRecord> readPatient(
        const std::vector<std::string>& fields, std::string* warning);

    // Puts the temporary file over the real file. The old file is only
    // removed when the new one is ready, so a failed write cannot destroy
    // the data that is already on the disk.
    static void replaceFile(const std::string& temporary, const std::string& filename);
};

} // namespace utils
} // namespace scapms
```

utils/IDGenerator.h

```
#pragma once

#include <string>
#include <vector>

namespace scapms {
namespace utils {

// Produces the identifiers the clinic hands out.
//
// The generator is stateless: it looks at the identifiers that are already in
// use and returns the next free one. That keeps the number generation in one
// place instead of spreading counters across the GUI.
class IDGenerator {
public:
    // Next free "APT-####" identifier.
    static std::string nextAppointmentId(const std::vector<std::string>& takenIds);

    // Next free identifier for a patient category, for example "GEN-106".
    static std::string nextPatientId(const std::string& prefix,
                                     const std::vector<std::string>& takenIds);

    static const char* patientPrefix(const std::string& patientType);

private:
    // Highest number already used for the given prefix, or 0 when none.
    static int highestUsedNumber(const std::string& prefix,
                                 const std::vector<std::string>& takenIds);
};

} // namespace utils
} // namespace scapms
```

utils/StringUtil.h

```
#pragma once

#include <algorithm>
#include <cctype>
#include <string>

namespace scapms {
namespace utils {

// Lower cases an ASCII string. Used by the name search so that the clinic
// staff does not have to remember the exact capitalisation.
inline std::string toLower(const std::string& text) {
    std::string lowered = text;
    std::transform(lowered.begin(), lowered.end(), lowered.begin(),
                   [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
    return lowered;
}

inline bool equalsIgnoreCase(const std::string& left, const std::string& right) {
    return toLower(left) == toLower(right);
}

// Orders two strings ignoring letter case. The result is less than zero
// when the first one comes first in the alphabet, and greater than zero
// when the second one comes first.
//
// The sort of the clinic uses this, so that a name that was typed in small
// letters is not put at the end of the list. The search already ignores
// the letter case, and the two have to agree.
inline int compareIgnoreCase(const std::string& left, const std::string& right) {
    const std::string loweredLeft = toLower(left);
    const std::string loweredRight = toLower(right);

    if (loweredLeft < loweredRight) {
        return -1;
    }
    if (loweredRight < loweredLeft) {
        return 1;
    }
    return 0;
}

// True when the text holds the part, ignoring letter case. An empty part
// never matches, so that a search for nothing finds nothing.
inline bool containsIgnoreCase(const std::string& text, const std::string& part) {
    if (part.empty()) {
        return false;
    }
    return toLower(text).find(toLower(part)) != std::string::npos;
}

inline std::string trim(const std::string& text) {
    std::size_t first = text.find_first_not_of(" \t\r\n");
    if (first == std::string::npos) {
        return "";
    }
    std::size_t last = text.find_last_not_of(" \t\r\n");
    return text.substr(first, last - first + 1);
}

} // namespace utils
} // namespace scapms
```

4.2 Source files

controller/AppointmentController.cpp

```
#include "AppointmentController.h"

#include "../model/Appointable.h"
#include "../model/PatientRecord.h"
#include "../utils/ClinicExceptions.h"
#include "../utils/DateUtil.h"
#include "../utils/IDGenerator.h"

namespace scapms {
namespace controller {

AppointmentController::AppointmentController(model::ClinicDatabase& database)
    : database(database) {}

void AppointmentController::checkBookingRules(const model::Appointment& appointment) const {
    if (appointment.getAppointmentID().empty()) {
        throw utils::InvalidAppointmentException("An appointment needs an identifier.");
    }
    if (appointment.getDate().empty()) {
        throw utils::InvalidAppointmentException(
            "An appointment needs a date, written as YYYY-MM-DD.");
    }
    if (appointment.getTime().empty()) {
        throw utils::InvalidAppointmentException(
            "An appointment needs a time, written as HH:MM AM or HH:MM PM.");
    }
    if (!utils::isValidDate(appointment.getDate())) {
        throw utils::InvalidAppointmentException(
            "The date [" + appointment.getDate() + "] is not a valid calendar date.");
    }
    if (utils::timeToMinutes(appointment.getTime()) < 0) {
        throw utils::InvalidAppointmentException(
            "The time [" + appointment.getTime() + "] is not a valid time of the day.");
    }
}

bool AppointmentController::patientCanBeBooked(const model::Appointment& appointment) const {
    const model::PatientRecord* patient = database.findPatientByID(appointment.getPatientID());
    if (patient == nullptr) {
        return false;
    }
    // Only the categories that answer the Appointable interface can be
    // booked. An emergency case is seen in the waiting room instead.
    return dynamic_cast<const model::Appointable*>(patient) != nullptr;
}

void AppointmentController::scheduleAppointment(const model::Appointment& appointment,
                                                bool trackUndo) {
    checkBookingRules(appointment);

    if (appointment.getPatientID().empty()) {
        throw utils::InvalidAppointmentException("An appointment needs a patient.");
    }

    if (database.findPatientByID(appointment.getPatientID()) == nullptr) {
        throw utils::InvalidAppointmentException(
            "No patient with ID [" + appointment.getPatientID() + "] is registered.");
    }

    if (!patientCanBeBooked(appointment)) {
        const model::PatientRecord* patient = database.findPatientByID(appointment.getPatientID());
        throw utils::InvalidAppointmentException(
            "A patient of the category [" + patient->getTypeLabel() +
            "] is not booked in advance. Please use the waiting room instead.");
    }

    // The same doctor cannot be in two places at the same time.
    const int newMinutes = utils::timeToMinutes(appointment.getTime());
    for (const model::Appointment& existing : database.getAllAppointments()) {
```

controller/ClinicManager.cpp

```
#include "ClinicManager.h"

#include "../utils/ClinicExceptions.h"
#include "../utils/FileHandler.h"
#include "../utils/IDGenerator.h"
#include "../model/GeneralPatient.h"

#include <cctype>

namespace scapms {
namespace controller {

ClinicManager::ClinicManager() : appointmentController(clinicDatabase) {}

ClinicManager::~ClinicManager() = default;

void ClinicManager::checkPatientRules(const model::PatientRecord& patient) const {
    if (patient.getAge() < 0 || patient.getAge() > 120) {
        throw utils::InvalidPatientDataException(
            "Patient age (" + std::to_string(patient.getAge()) +
            ") must be between 0 and 120 years.");
    }
    if (patient.getName().empty()) {
        throw utils::InvalidPatientDataException("Patient name cannot be empty.");
    }
    if (patient.getPatientID().empty()) {
        throw utils::InvalidPatientDataException("Patient ID cannot be empty.");
    }

    // The contact number is checked when it is given. It may be left empty,
    // because the clinic does not always have it, but when it is written it
    // has to look like a telephone number.
    const std::string& contact = patient.getContactNumber();
    if (!contact.empty()) {
        bool allowed = false;
        for (char c : contact) {
            if (std::isdigit(static_cast<unsigned char>(c)) || c == '+' || c == '-' ||
                c == ' ' || c == '(' || c == ')') {
                allowed = true;
            } else {
                allowed = false;
                break;
            }
        }
        if (!allowed) {
            throw utils::InvalidPatientDataException(
                "The contact number [" + contact +
                "] may only hold digits and the characters + - ( ) and spaces.");
        }
    }

    // The identifier of a patient carries the category, so that a general
    // patient is GEN-101, an emergency case is EMG-999 and a follow-up
    // patient is FOL-205. A record that breaks the rule is refused, so
    // that the list is not mixed up later on.
    const std::string expectedPrefix = utils::IDGenerator::patientPrefix(patient.getPatientType());
    if (patient.getPatientID().compare(0, expectedPrefix.size(), expectedPrefix) != 0) {
        throw utils::InvalidPatientDataException(
            "The ID of a patient of the category [" + patient.getTypeLabel() +
            "] has to begin with [" + expectedPrefix + "], for example [" +
            expectedPrefix + "001].");
    }

    // The consultation fee has to be a real amount. A value of "nan" or of
    // infinity would pass every comparison, because every comparison with
    // it is false, and it would then be written into the data file.
    const model::GeneralPatient* general =
        dynamic_cast<const model::GeneralPatient*>(&patient);
    if (general != nullptr &&
```

controller/ReportBuilder.cpp

```
#include "ReportBuilder.h"

#include <ctime>
#include <iomanip>
#include <map>
#include <sstream>

namespace scapms {
namespace controller {

namespace {

const char* RULE = "------------------------------------------------------------";

// Pads a value to the given width. A negative width keeps the value on the
// left and fills the rest with spaces.
std::string pad(const std::string& value, int width) {
    const std::size_t target = (width < 0) ? static_cast<std::size_t>(-width)
                                           : static_cast<std::size_t>(width);

    std::string padded = value;
    while (padded.size() < target) {
        padded += " ";
    }
    return padded;
}

std::string padNumber(int value, int width) {
    std::ostringstream out;
    out << value;
    return pad(out.str(), width);
}

// One line of a report: a label and a number.
std::string line(const std::string& label, int value) {
    return pad(label, 28) + padNumber(value, 5) + "\n";
}

} // namespace

std::string ReportBuilder::timestamp() {
    std::time_t now = std::time(nullptr);
    std::tm parts = {};
    localtime_s(&parts, &now);

    char buffer[32] = {0};
    std::strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", &parts);
    return std::string(buffer);
}

std::string ReportBuilder::header(const std::string& title) {
    std::string out = "CLINIC REPORT\n";
    out += title + "\n";
    out += "Generated: " + timestamp() + "\n";
    out += RULE;
    out += "\n";
    return out;
}

std::string ReportBuilder::categoryDistribution(const ClinicManager& manager) {
    int generalCount = 0;
    int emergencyCount = 0;
    int followUpCount = 0;

    for (const model::PatientRecord* patient : manager.getAllPatients()) {
        if (patient == nullptr) {
            continue;
        }
        if (patient->getPatientType() == model::PatientRecord::TYPE_GENERAL) {
```

controller/SearchEngine.cpp

```
#include "SearchEngine.h"

#include "../utils/StringUtil.h"

namespace scapms {
namespace controller {

int SearchEngine::linearSearchByID(const std::vector<model::PatientRecord*>& records,
                                   const std::string& targetID) {
    for (int i = 0; i < static_cast<int>(records.size()); ++i) {
        if (records[i] != nullptr && records[i]->getPatientID() == targetID) {
            return i;
        }
    }
    return -1;
}

std::vector<int> SearchEngine::linearSearchByText(
    const std::vector<model::PatientRecord*>& records, const std::string& text) {
    std::vector<int> matches;

    if (text.empty()) {
        return matches;                          // a search for nothing finds nothing
    }

    for (int i = 0; i < static_cast<int>(records.size()); ++i) {
        if (records[i] == nullptr) {
            continue;
        }
        // The identifier and the name are both looked at, because the
        // staff may type either one.
        if (utils::containsIgnoreCase(records[i]->getPatientID(), text) ||
            utils::containsIgnoreCase(records[i]->getName(), text)) {
            matches.push_back(i);
        }
    }
    return matches;
}

int SearchEngine::binarySearchByID(const std::vector<model::PatientRecord*>& sortedRecords,
                                   const std::string& targetID) {
    int low = 0;
    int high = static_cast<int>(sortedRecords.size()) - 1;

    while (low <= high) {
        // The middle of the range, written so that it cannot overflow.
        int middle = low + (high - low) / 2;

        const std::string& middleID = sortedRecords[middle]->getPatientID();

        // The comparison is the same one the sort uses, ignoring the letter
        // case. If the two were different, the list would not really be
        // sorted in the way the binary search expects.
        const int order = utils::compareIgnoreCase(middleID, targetID);

        if (order == 0) {
            return middle;
        } else if (order < 0) {
            low = middle + 1;                    // look in the right half
        } else {
            high = middle - 1;                   // look in the left half
        }
    }
    return -1;
}

std::vector<int> SearchEngine::recursiveSearchByName(
    const std::vector<model::PatientRecord*>& records, const std::string& targetName,
    int currentIndex) {
```

controller/SortEngine.cpp

```
#include "SortEngine.h"

#include "../utils/StringUtil.h"

namespace scapms {
namespace controller {

bool SortEngine::comparePatients(const model::PatientRecord* first,
                                 const model::PatientRecord* second,
                                 SortField field) {
    if (first == nullptr || second == nullptr) {
        return false;
    }

    switch (field) {
        case SortField::PatientID:
            // The letter case is ignored, so that a name that was typed in
            // small letters does not jump to the end of the list.
            return utils::compareIgnoreCase(first->getPatientID(),
                                            second->getPatientID()) < 0;
        case SortField::PatientName:
            return utils::compareIgnoreCase(first->getName(),
                                            second->getName()) < 0;
        case SortField::PatientAge:
            return first->getAge() < second->getAge();
        default:
            return utils::compareIgnoreCase(first->getPatientID(),
                                            second->getPatientID()) < 0;
    }
}

void SortEngine::selectionSort(std::vector<model::PatientRecord*>& records, SortField field) {
    const int count = static_cast<int>(records.size());

    for (int i = 0; i < count - 1; ++i) {
        int smallestIndex = i;

        // Find the smallest record in the part that is not sorted yet.
        for (int j = i + 1; j < count; ++j) {
            if (comparePatients(records[j], records[smallestIndex], field)) {
                smallestIndex = j;
            }
        }

        if (smallestIndex != i) {
            model::PatientRecord* temp = records[i];
            records[i] = records[smallestIndex];
            records[smallestIndex] = temp;
        }
    }
}

void SortEngine::mergeSort(std::vector<model::PatientRecord*>& records, SortField field) {
    if (records.size() <= 1) {
        return;
    }
    mergeSortRecursive(records, 0, static_cast<int>(records.size()) - 1, field);
}

void SortEngine::mergeSortRecursive(std::vector<model::PatientRecord*>& records,
                                    int left, int right, SortField field) {
    if (left >= right) {
        return;                                 // one record is already sorted
    }

    const int middle = left + (right - left) / 2;

    mergeSortRecursive(records, left, middle, field);
    mergeSortRecursive(records, middle + 1, right, field);
```

model/AdminAction.cpp

```
#include "AdminAction.h"

namespace scapms {
namespace model {

AdminAction::AdminAction(Type actionType, const std::string& text,
                         const std::string& affectedPatientID)
    : type(actionType),
      description(text),
      patientID(affectedPatientID),
      patientBackup(nullptr),
      appointmentBackup("", "", "", "", ""),
      waitingQueuePosition(-1) {}

AdminAction::AdminAction(Type actionType, const std::string& text,
                         const std::string& affectedPatientID,
                         std::unique_ptr<PatientRecord> backup)
    : type(actionType),
      description(text),
      patientID(affectedPatientID),
      patientBackup(std::move(backup)),
      appointmentBackup("", "", "", "", ""),
      waitingQueuePosition(-1) {}

AdminAction::AdminAction(Type actionType, const std::string& text,
                         const std::string& affectedPatientID,
                         const Appointment& appointment)
    : type(actionType),
      description(text),
      patientID(affectedPatientID),
      patientBackup(nullptr),
      appointmentBackup(appointment),
      waitingQueuePosition(-1) {}

} // namespace model
} // namespace scapms
```

model/Appointment.cpp

```
#include "Appointment.h"

#include <iostream>

#include "../utils/DateUtil.h"

namespace scapms {
namespace model {

bool Appointment::happensBefore(const Appointment& first, const Appointment& second) {
    if (first.getDate() != second.getDate()) {
        return first.getDate() < second.getDate();
    }

    const int firstMinutes = utils::timeToMinutes(first.getTime());
    const int secondMinutes = utils::timeToMinutes(second.getTime());
    if (firstMinutes < 0 || secondMinutes < 0) {
        return first.getTime() < second.getTime();
    }
    return firstMinutes < secondMinutes;
}

void Appointment::displayAppointment() const {
    std::cout << "Appointment ID: [" << appointmentID << "] | Patient: " << patientID
              << " | Date: " << date << " " << time
              << " | Doctor: " << doctorName
              << " | Status: " << status << std::endl;
}

} // namespace model
} // namespace scapms
```

model/ClinicDatabase.cpp

```
#include "ClinicDatabase.h"

#include <algorithm>
#include <iostream>

#include "../utils/ClinicExceptions.h"

namespace scapms {
namespace model {

ClinicDatabase::ClinicDatabase() : recentCount(0) {
    for (int i = 0; i < CACHE_SIZE; ++i) {
        recentPatients[i] = nullptr;
    }
}

ClinicDatabase::~ClinicDatabase() {
    clearAll();
}

void ClinicDatabase::addPatient(std::unique_ptr<PatientRecord> patient) {
    addPatientInternal(std::move(patient), true);
}

void ClinicDatabase::addPatientWithoutUndo(std::unique_ptr<PatientRecord> patient) {
    addPatientInternal(std::move(patient), false);
}

void ClinicDatabase::addPatientInternal(std::unique_ptr<PatientRecord> patient, bool trackUndo) {
    if (patient == nullptr) {
        throw utils::InvalidPatientDataException("Cannot register a null patient record.");
    }

    // The rule that a patient identifier is used only once belongs to the
    // data itself, so it is kept here. The other rules of the clinic are
    // checked by the controller before it calls this method.
    if (findPatientByID(patient->getPatientID()) != nullptr) {
        throw utils::DuplicateIDException(patient->getPatientID());
    }

    const std::string patientID = patient->getPatientID();
    patients.push_back(std::move(patient));
    touchRecentPatient(patients.back().get());

    if (trackUndo) {
        // Undoing this removes the record again, so no copy is needed here.
        undoStack.push(AdminAction(AdminAction::Type::AddPatient,
                                   "Add Patient " + patientID,
                                   patientID));
    }
}

bool ClinicDatabase::removePatientInternal(const std::string& id,
                                           std::unique_ptr<PatientRecord>* backupOut,
                                           int* waitingPositionOut) {
    for (auto it = patients.begin(); it != patients.end(); ++it) {
        if ((*it)->getPatientID() != id) {
            continue;
        }

        PatientRecord* target = it->get();

        if (backupOut != nullptr) {
            // The undo stack owns this copy.
            backupOut->reset(target->clone());
        }

        if (waitingPositionOut != nullptr) {
            // The place of the record in the waiting queue is kept, so that
```

model/EmergencyPatient.cpp

```
#include "EmergencyPatient.h"

#include <sstream>

namespace scapms {
namespace model {

std::string EmergencyPatient::getUrgencyDescription() const {
    switch (severityLevel) {
        case 1:  return "CRITICAL - Immediate Attention Required!";
        case 2:  return "URGENT - Attend within 15 minutes";
        default: return "MODERATE - Priority waiting";
    }
}

std::string EmergencyPatient::getPatientReport() const {
    std::ostringstream out;
    out << "-------------------------------------\n";
    out << "[CATEGORY: EMERGENCY - HIGH PRIORITY]\n";
    out << "Patient ID:     " << patientID << "\n";
    out << "Name:           " << name << "\n";
    out << "Age:            " << age << "\n";
    out << "Contact:        " << contactNumber << "\n";
    out << "Severity (1-3): " << severityLevel
        << " (" << getUrgencyDescription() << ")\n";
    out << "Triage Note:    " << triageNote << "\n";
    out << "-------------------------------------";
    return out.str();
}

std::string EmergencyPatient::getSpecialDetail() const {
    return "Severity " + std::to_string(severityLevel) + " (" + triageNote + ")";
}

void EmergencyPatient::restoreFrom(const PatientRecord& previous) {
    PatientRecord::restoreFrom(previous);

    // The dynamic cast checks that the snapshot really is an emergency
    // patient. A static cast would be undefined behaviour when it is not.
    if (const EmergencyPatient* snapshot = dynamic_cast<const EmergencyPatient*>(&previous)) {
        severityLevel = snapshot->severityLevel;
        triageNote = snapshot->triageNote;
    }
}

} // namespace model
} // namespace scapms
```

model/FollowUpPatient.cpp

```
#include "FollowUpPatient.h"

#include <sstream>

namespace scapms {
namespace model {

std::string FollowUpPatient::getPatientReport() const {
    std::ostringstream out;
    out << "-------------------------------------\n";
    out << "[Category: Follow-Up Consultation]\n";
    out << "Patient ID:       " << patientID << "\n";
    out << "Name:             " << name << "\n";
    out << "Age:              " << age << "\n";
    out << "Contact:          " << contactNumber << "\n";
    out << "Last Visit Date:  " << previousVisitDate << "\n";
    out << "Assigned Doctor:  " << assignedDoctor << "\n";
    out << "Appointment:      " << getAppointmentDetails() << "\n";
    out << "-------------------------------------";
    return out.str();
}

std::string FollowUpPatient::getSpecialDetail() const {
    return "Follow-up with " + assignedDoctor + " (Last: " + previousVisitDate + ")";
}

void FollowUpPatient::restoreFrom(const PatientRecord& previous) {
    ScheduledPatient::restoreFrom(previous);

    // The dynamic cast checks that the snapshot really is a follow-up
    // patient. A static cast would be undefined behaviour when it is not.
    if (const FollowUpPatient* snapshot = dynamic_cast<const FollowUpPatient*>(&previous)) {
        previousVisitDate = snapshot->previousVisitDate;
        assignedDoctor = snapshot->assignedDoctor;
    }
}

} // namespace model
} // namespace scapms
```

model/GeneralPatient.cpp

```
#include "GeneralPatient.h"

#include <iomanip>
#include <sstream>

namespace scapms {
namespace model {

namespace {

// The clinic quotes its fees in naira.
std::string formatFee(double fee) {
    std::ostringstream out;
    out << "N" << std::fixed << std::setprecision(2) << fee;
    return out.str();
}

} // namespace

std::string GeneralPatient::getPatientReport() const {
    std::ostringstream out;
    out << "-------------------------------------\n";
    out << "[Category: General Outpatient]\n";
    out << "Patient ID:       " << patientID << "\n";
    out << "Name:             " << name << "\n";
    out << "Age:              " << age << "\n";
    out << "Contact:          " << contactNumber << "\n";
    out << "Consultation Fee: " << formatFee(consultationFee) << "\n";
    out << "Appointment:      " << getAppointmentDetails() << "\n";
    out << "-------------------------------------";
    return out.str();
}

std::string GeneralPatient::getSpecialDetail() const {
    return "Fee: " + formatFee(consultationFee);
}

double GeneralPatient::getBillAmount() const {
    return consultationFee;
}

std::string GeneralPatient::describeBill() const {
    return "Consultation fee: " + formatFee(consultationFee);
}

void GeneralPatient::restoreFrom(const PatientRecord& previous) {
    ScheduledPatient::restoreFrom(previous);

    // The dynamic cast checks that the snapshot really is a general patient.
    // A static cast would be undefined behaviour when it is not.
    if (const GeneralPatient* snapshot = dynamic_cast<const GeneralPatient*>(&previous)) {
        consultationFee = snapshot->consultationFee;
    }
}

} // namespace model
} // namespace scapms
```

model/PatientRecord.cpp

```
#include "PatientRecord.h"

#include <iostream>

namespace scapms {
namespace model {

const Appointment* PatientRecord::findAppointment(const std::string& appointmentID) const {
    for (const Appointment& appointment : appointments) {
        if (appointment.getAppointmentID() == appointmentID) {
            return &appointment;
        }
    }
    return nullptr;
}

bool PatientRecord::hasAppointment(const std::string& appointmentID) const {
    return findAppointment(appointmentID) != nullptr;
}

Appointment* PatientRecord::findAppointmentForUpdate(const std::string& appointmentID) {
    for (Appointment& appointment : appointments) {
        if (appointment.getAppointmentID() == appointmentID) {
            return &appointment;
        }
    }
    return nullptr;
}

const Appointment* PatientRecord::getNextPendingAppointment() const {
    const Appointment* earliest = nullptr;

    for (const Appointment& appointment : appointments) {
        if (appointment.getStatus() != Appointment::STATUS_PENDING) {
            continue;
        }
        if (earliest == nullptr || Appointment::happensBefore(appointment, *earliest)) {
            earliest = &appointment;
        }
    }
    return earliest;
}

std::size_t PatientRecord::getPendingAppointmentCount() const {
    std::size_t pending = 0;
    for (const Appointment& appointment : appointments) {
        if (appointment.getStatus() == Appointment::STATUS_PENDING) {
            ++pending;
        }
    }
    return pending;
}

bool PatientRecord::removeAppointment(const std::string& appointmentID) {
    for (std::size_t i = 0; i < appointments.size(); ++i) {
        if (appointments[i].getAppointmentID() == appointmentID) {
            appointments.erase(appointments.begin() + static_cast<std::ptrdiff_t>(i));
            return true;
        }
    }
    return false;
}

bool PatientRecord::restoreAppointment(const Appointment& backup) {
    for (Appointment& appointment : appointments) {
        if (appointment.getAppointmentID() == backup.getAppointmentID()) {
            appointment.setStatus(backup.getStatus());
            return true;
        }
```

model/ScheduledPatient.cpp

```
#include "ScheduledPatient.h"

namespace scapms {
namespace model {

bool ScheduledPatient::scheduleAppointment(const Appointment& appointment) {
    if (appointment.getDate().empty() || appointment.getTime().empty()) {
        return false;                          // a booking needs a date and a time
    }
    return addAppointment(appointment);
}

bool ScheduledPatient::cancelAppointment() {
    const Appointment* next = getNextPendingAppointment();
    if (next == nullptr) {
        return false;                          // there is no booking to cancel
    }
    return cancelAppointment(next->getAppointmentID());
}

bool ScheduledPatient::cancelAppointment(const std::string& appointmentID) {
    Appointment* target = findAppointmentForUpdate(appointmentID);
    if (target == nullptr) {
        return false;                          // the patient does not hold that booking
    }
    if (target->getStatus() != Appointment::STATUS_PENDING) {
        return false;                          // it was cancelled or completed already
    }
    target->setStatus(Appointment::STATUS_CANCELLED);
    return true;
}

bool ScheduledPatient::hasScheduledAppointment() const {
    return getPendingAppointmentCount() > 0;
}

std::string ScheduledPatient::getAppointmentDetails() const {
    const Appointment* next = getNextPendingAppointment();
    if (next == nullptr) {
        return "No active appointment";
    }
    return next->getDate() + " at " + next->getTime();
}

double ScheduledPatient::getBillAmount() const {
    return 0.0;                                // this category does not charge
}

std::string ScheduledPatient::describeBill() const {
    return "No charge is recorded for this category.";
}

} // namespace model
} // namespace scapms
```

utils/DateUtil.cpp

```
#include "DateUtil.h"

#include <cstdio>
#include <cstdlib>

namespace scapms {
namespace utils {

int timeToMinutes(const std::string& time) {
    // Expected shape: "HH:MM AM" or "HH:MM PM".
    int hour = 0;
    int minute = 0;
    char meridiem[3] = {0, 0, 0};

    if (std::sscanf(time.c_str(), "%d:%d %2s", &hour, &minute, meridiem) != 3) {
        return -1;
    }
    if (hour < 0 || hour > 23 || minute < 0 || minute > 59) {
        return -1;
    }

    // The marker has to be a real AM or PM. Without this check a time such
    // as "10:30 XM" would be accepted and stored as bad data.
    if (meridiem[0] != 'A' && meridiem[0] != 'P') {
        return -1;
    }
    if (meridiem[1] != 'M') {
        return -1;
    }

    // With a marker the hour runs from 1 to 12.
    if (hour < 1 || hour > 12) {
        return -1;
    }

    int total = hour * 60 + minute;
    if (meridiem[0] == 'P' && hour < 12) {
        total += 12 * 60;                       // afternoon
    } else if (meridiem[0] == 'A' && hour == 12) {
        total -= 12 * 60;                       // midnight
    }
    return total;
}

bool isValidDate(const std::string& date) {
    int year = 0;
    int month = 0;
    int day = 0;

    if (std::sscanf(date.c_str(), "%d-%d-%d", &year, &month, &day) != 3) {
        return false;
    }
    if (year < 1900 || year > 2200 || month < 1 || month > 12 || day < 1 || day > 31) {
        return false;
    }

    // Reject the impossible days such as 31 April.
    static const int daysInMonth[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    int limit = daysInMonth[month - 1];
    if (month == 2 && ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)) {
        limit = 29;                             // leap year
    }
    return day <= limit;
}

} // namespace utils
} // namespace scapms
```

utils/FileHandler.cpp

```
#include "FileHandler.h"

#include <cstdio>
#include <fstream>
#include <iomanip>
#include <memory>
#include <ostream>
#include <sstream>

#include "StringUtil.h"
#include "../model/GeneralPatient.h"
#include "../model/EmergencyPatient.h"
#include "../model/FollowUpPatient.h"

namespace scapms {
namespace utils {

namespace {

// Reads a whole number. Returns false when the text is not a number.
bool parseInt(const std::string& text, int* value) {
    if (text.empty()) {
        return false;
    }
    try {
        std::size_t used = 0;
        int parsed = std::stoi(text, &used);
        if (used != text.size()) {
            return false;
        }
        *value = parsed;
        return true;
    } catch (const std::exception&) {
        return false;
    }
}

// Reads a decimal number. Returns false when the text is not a number.
bool parseDouble(const std::string& text, double* value) {
    if (text.empty()) {
        return false;
    }
    try {
        std::size_t used = 0;
        double parsed = std::stod(text, &used);
        if (used != text.size()) {
            return false;
        }
        *value = parsed;
        return true;
    } catch (const std::exception&) {
        return false;
    }
}

} // namespace

std::vector<std::string> FileHandler::splitRow(const std::string& line) {
    std::vector<std::string> fields;
    std::string current;
    bool insideQuotes = false;

    for (std::size_t i = 0; i < line.size(); ++i) {
        char c = line[i];

        if (insideQuotes) {
            if (c == '"') {
                if (i + 1 < line.size() && line[i + 1] == '"') {
                    current += '"';
```

utils/IDGenerator.cpp

```
#include "IDGenerator.h"

#include <cerrno>
#include <climits>
#include <cstdlib>

#include "../model/PatientRecord.h"

namespace scapms {
namespace utils {

const char* IDGenerator::patientPrefix(const std::string& patientType) {
    if (patientType == model::PatientRecord::TYPE_EMERGENCY) {
        return "EMG-";
    }
    if (patientType == model::PatientRecord::TYPE_FOLLOWUP) {
        return "FOL-";
    }
    return "GEN-";
}

int IDGenerator::highestUsedNumber(const std::string& prefix,
                                   const std::vector<std::string>& takenIds) {
    int highest = 0;

    for (const std::string& id : takenIds) {
        if (id.size() <= prefix.size() || id.compare(0, prefix.size(), prefix) != 0) {
            continue;                           // different prefix
        }

        std::string digits = id.substr(prefix.size());
        if (digits.find_first_not_of("0123456789") != std::string::npos) {
            continue;                           // not a plain number
        }
        // The number is read with strtol and not with atoi, because atoi has
        // undefined behaviour when the number does not fit into an int. A
        // file with a very long number would otherwise produce a nonsense
        // identifier such as [GEN--2147483648].
        errno = 0;
        char* end = nullptr;
        const long number = std::strtol(digits.c_str(), &end, 10);
        if (errno == ERANGE || end == digits.c_str() ||
            (end != nullptr && *end != '\0')) {
            continue;                           // too large to be a usable number
        }
        if (number > highest && number < INT_MAX) {
            highest = static_cast<int>(number);
        }
    }
    return highest;
}

std::string IDGenerator::nextAppointmentId(const std::vector<std::string>& takenIds) {
    const std::string prefix = "APT-";
    int next = highestUsedNumber(prefix, takenIds) + 1;

    // Keep looking in the unlikely case that the number is already taken by
    // an identifier that does not follow the numeric pattern.
    while (true) {
        std::string candidate = prefix + std::to_string(next);
        bool taken = false;
        for (const std::string& id : takenIds) {
            if (id == candidate) {
                taken = true;
                break;
            }
        }
        if (!taken) {
            return candidate;
```

4.3 GUI files and resources

SCAPMS.pro

```
QT += core gui widgets

TARGET = SCAPMS_GUI
TEMPLATE = app

CONFIG += c++17
DEFINES += _CRT_SECURE_NO_WARNINGS

# The graphical user interface.
#
# The program has two entry points: gui_main.cpp builds the graphical
# program, and main.cpp builds the console demonstration of the same model,
# controller and utility layers. They cannot be in one binary, because each
# of them has its own main(), so the console version is built by build.bat
# with the compiler directly and is not part of this project file.
SOURCES += \
    gui_main.cpp \
    gui/MainWindow.cpp \
    gui/PatientPanel.cpp \
    gui/AppointmentPanel.cpp \
    gui/QueuePanel.cpp \
    gui/SearchPanel.cpp \
    gui/ReportPanel.cpp \
    gui/AdminPanel.cpp \

HEADERS += \
    gui/MainWindow.h \
    gui/PatientPanel.h \
    gui/AppointmentPanel.h \
    gui/QueuePanel.h \
    gui/SearchPanel.h \
    gui/ReportPanel.h \
    gui/AdminPanel.h \

# The model layer.
SOURCES += \
    model/PatientRecord.cpp \
    model/ScheduledPatient.cpp \
    model/GeneralPatient.cpp \
    model/EmergencyPatient.cpp \
    model/FollowUpPatient.cpp \
    model/Appointment.cpp \
    model/AdminAction.cpp \
    model/ClinicDatabase.cpp

HEADERS += \
    model/PatientRecord.h \
    model/Appointable.h \
    model/Billable.h \
    model/ScheduledPatient.h \
    model/GeneralPatient.h \
    model/EmergencyPatient.h \
    model/FollowUpPatient.h \
    model/Appointment.h \
    model/AdminAction.h \
    model/ClinicDatabase.h

# The controller layer.
SOURCES += \
    controller/ClinicManager.cpp \
    controller/AppointmentController.cpp \
    controller/SearchEngine.cpp \
    controller/SortEngine.cpp \
    controller/ReportBuilder.cpp

HEADERS += \
    controller/ClinicManager.h \
    controller/AppointmentController.h \
    controller/SearchEngine.h \
```

build.bat

```
@echo off
REM Builds the console demonstration of SCAPMS (no Qt required).
REM The console build proves that the searching, sorting, queue, undo and
REM reporting features work on their own.
REM The four reports of the clinic are printed at the end of the run.

setlocal
call "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat" x64 >nul 2>&1
cd /d "%~dp0"

if not exist build mkdir build

echo [1/2] Compiling the console demonstration...
cl /nologo /EHsc /std:c++17 /W3 /D_CRT_SECURE_NO_WARNINGS /Fe:build\SCAPMS.exe ^
   main.cpp ^
   model\PatientRecord.cpp ^
   model\ScheduledPatient.cpp ^
   model\GeneralPatient.cpp ^
   model\EmergencyPatient.cpp ^
   model\FollowUpPatient.cpp ^
   model\Appointment.cpp ^
   model\AdminAction.cpp ^
   model\ClinicDatabase.cpp ^
   controller\ClinicManager.cpp ^
   controller\AppointmentController.cpp ^
   controller\SearchEngine.cpp ^
   controller\SortEngine.cpp ^
   controller\ReportBuilder.cpp ^
   utils\DateUtil.cpp ^
   utils\FileHandler.cpp ^
   utils\IDGenerator.cpp ^
   /Fo:build\ /Fd:build\vc.pdb

if errorlevel 1 (
    echo.
    echo Build FAILED
    exit /b 1
)

echo [2/2] Running the demonstration...
echo.
build\SCAPMS.exe

endlocal
```

build_gui.bat

```
@echo off
setlocal
call "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvarsall.bat" x64 >nul 2>&1
set PATH=C:\Qt\6.5.3\msvc2019_64\bin;%PATH%
cd /d "%~dp0"

echo [1/3] Generating Makefile with qmake...
C:\Qt\6.5.3\msvc2019_64\bin\qmake.exe SCAPMS.pro -spec win32-msvc "CONFIG+=release"
if errorlevel 1 goto :failed

echo [2/3] Compiling Qt Application with nmake...
nmake /NOLOGO release
if errorlevel 1 goto :failed

echo [3/3] Deploying Qt runtime DLLs...
C:\Qt\6.5.3\msvc2019_64\bin\windeployqt.exe release\SCAPMS_GUI.exe --no-translations --compiler-runtime >nul 2>&1
if errorlevel 1 goto :failed

echo.
echo Build Complete! Binary created at: release\SCAPMS_GUI.exe
exit /b 0

:failed
echo.
echo Build FAILED
exit /b 1
```

gui/AdminPanel.cpp

```
#include "AdminPanel.h"

#include <QDir>
#include <QFileDialog>
#include <QFileInfo>
#include <QFormLayout>
#include <QGridLayout>
#include <QGroupBox>
#include <QHBoxLayout>
#include <QLabel>
#include <QMessageBox>
#include <QVBoxLayout>

#include <vector>

#include "../model/Appointment.h"
#include "../utils/ClinicExceptions.h"

namespace scapms {
namespace gui {

AdminPanel::AdminPanel(controller::ClinicManager& manager, QWidget* parent)
    : QWidget(parent), manager(manager) {
    QVBoxLayout* layout = new QVBoxLayout(this);
    layout->setContentsMargins(16, 16, 16, 16);
    layout->setSpacing(12);

    QGroupBox* steps = new QGroupBox("Administrative Operations", this);
    QHBoxLayout* stepRow = new QHBoxLayout(steps);
    stepRow->setSpacing(10);

    btnUndo = new QPushButton("Undo Last Operation", steps);
    btnUndo->setToolTip("Reverse the last change to a patient or an appointment.");
    connect(btnUndo, &QPushButton::clicked, this, &AdminPanel::onUndoClicked);

    btnSave = new QPushButton("Save Data", steps);
    btnSave->setToolTip("Write the patients and the appointments to CSV files.");
    connect(btnSave, &QPushButton::clicked, this, &AdminPanel::onSaveClicked);

    btnLoad = new QPushButton("Load Data", steps);
    btnLoad->setToolTip("Read the patients and the appointments from CSV files.");
    connect(btnLoad, &QPushButton::clicked, this, &AdminPanel::onLoadClicked);

    btnNew = new QPushButton("New Database", steps);
    btnNew->setToolTip("Clear every record and start again with an empty clinic.");
    connect(btnNew, &QPushButton::clicked, this, &AdminPanel::onNewClicked);

    stepRow->addWidget(btnUndo);
    stepRow->addWidget(btnSave);
    stepRow->addWidget(btnLoad);
    stepRow->addStretch();
    stepRow->addWidget(btnNew);

    layout->addWidget(steps);

    QHBoxLayout* lowerRow = new QHBoxLayout();
    lowerRow->setSpacing(12);

    QGroupBox* dashboardBox = new QGroupBox("Clinic Dashboard", this);
    QVBoxLayout* dashboardLayout = new QVBoxLayout(dashboardBox);
    dashboardLayout->setContentsMargins(16, 20, 16, 16);
    dashboardLayout->setSpacing(14);

    // The numbers of the clinic are put in a grid, so that the dashboard is
    // made of real widgets and the figures line up under each other.
    QGridLayout* figures = new QGridLayout();
    figures->setHorizontalSpacing(18);
    figures->setVerticalSpacing(10);
```

gui/AdminPanel.h

```
#pragma once

#include <QWidget>
#include <QLabel>
#include <QListWidget>
#include <QPushButton>

#include "../controller/ClinicManager.h"

namespace scapms {
namespace gui {

// The administration panel. It holds the operations that belong to the
// staff in charge of the clinic rather than to the treatment of a patient:
// undoing the last step, saving and loading the data, and starting a new
// day with an empty database.
//
// The panel also shows the dashboard of the clinic and the records that were
// used most recently.
class AdminPanel : public QWidget {
    Q_OBJECT

public:
    explicit AdminPanel(controller::ClinicManager& manager, QWidget* parent = nullptr);

    void refresh();

    // Redraws only the statistics of the dashboard. The timer of the window
    // calls this, so that the numbers of the clinic are always the true
    // ones even when nothing was changed.
    void refreshStatistics();

    // The administrative steps. The window connects its menu entries to
    // these, so that the menu and the panel do the same thing.
    void undoLastOperation();
    void saveData();
    void loadData();
    void newDatabase();

signals:
    void dataChanged();

private slots:
    void onUndoClicked();
    void onSaveClicked();
    void onLoadClicked();
    void onNewClicked();

private:
    controller::ClinicManager& manager;

    QPushButton* btnUndo;
    QPushButton* btnSave;
    QPushButton* btnLoad;
    QPushButton* btnNew;

    // The numbers of the dashboard. Every one of them is a label, so the
    // dashboard is built out of real widgets and not out of one block of
    // text.
    QLabel* lblPatientCount;
    QLabel* lblWaitingCount;
    QLabel* lblBookedCount;
    QLabel* lblCompletedCount;
    QLabel* lblPendingCount;
    QLabel* lblCancelledCount;

    QListWidget* listRecent;

    // A small label in the colour of one category, used by the table of the
```

gui/AppointmentPanel.cpp

```
#include "AppointmentPanel.h"

#include <QColor>
#include <QDate>
#include <QFormLayout>
#include <QGroupBox>
#include <QHBoxLayout>
#include <QHeaderView>
#include <QMessageBox>
#include <QVBoxLayout>

#include "../model/Appointment.h"
#include "../model/Appointable.h"
#include "../utils/ClinicExceptions.h"
#include "../utils/DateUtil.h"

namespace scapms {
namespace gui {

AppointmentPanel::AppointmentPanel(controller::ClinicManager& manager, QWidget* parent)
    : QWidget(parent), manager(manager), aptIdEdited(false) {
    QHBoxLayout* layout = new QHBoxLayout(this);
    layout->setContentsMargins(16, 16, 16, 16);
    layout->setSpacing(16);

    layout->addWidget(buildForm(this));
    layout->addWidget(buildSchedule(this));

    refresh();
}

QWidget* AppointmentPanel::buildForm(QWidget* parent) {
    QGroupBox* group = new QGroupBox("Schedule New Appointment", parent);
    QFormLayout* form = new QFormLayout(group);
    form->setContentsMargins(16, 20, 16, 16);
    form->setVerticalSpacing(12);
    form->setLabelAlignment(Qt::AlignLeft | Qt::AlignVCenter);
    form->setFieldGrowthPolicy(QFormLayout::FieldsStayAtSizeHint);

    txtAptID = new QLineEdit(group);
    txtAptID->setToolTip("The identifier of the booking. A free one is filled in for you.");

    cmbAptPatientID = new QComboBox(group);
    cmbAptPatientID->setToolTip("The patient that the appointment is for.");

    txtAptDate = new QLineEdit("2026-11-25", group);
    txtAptDate->setPlaceholderText("YYYY-MM-DD");

    txtAptTime = new QLineEdit("10:00 AM", group);
    txtAptTime->setPlaceholderText("HH:MM AM / PM");

    txtAptDoctor = new QLineEdit("Dr. Adeyemi", group);

    // The inputs keep a fixed width, so that the form never squeezes the
    // schedule table on a small screen.
    const int inputWidth = 190;
    for (QLineEdit* input : {txtAptID, txtAptDate, txtAptTime, txtAptDoctor}) {
        input->setMaximumWidth(inputWidth);
    }
    cmbAptPatientID->setMaximumWidth(inputWidth);

    form->addRow("Appointment ID:", txtAptID);
    form->addRow("Patient:", cmbAptPatientID);
    form->addRow("Date:", txtAptDate);
    form->addRow("Time:", txtAptTime);
    form->addRow("Doctor:", txtAptDoctor);

    // A free identifier is filled in, unless the staff types one of their own.
    connect(txtAptID, &QLineEdit::textEdited, this, [this]() { aptIdEdited = true; });
```

gui/AppointmentPanel.h

```
#pragma once

#include <QWidget>
#include <QComboBox>
#include <QLineEdit>
#include <QPushButton>
#include <QTableWidget>

#include "../controller/ClinicManager.h"

namespace scapms {
namespace gui {

// The appointment panel. It holds the booking form and the schedule table.
class AppointmentPanel : public QWidget {
    Q_OBJECT

public:
    explicit AppointmentPanel(controller::ClinicManager& manager, QWidget* parent = nullptr);

    void refresh();

signals:
    void dataChanged();

private slots:
    void onScheduleAppointment();
    void onCancelSelectedAppointment();
    void onCompleteSelectedAppointment();

private:
    controller::ClinicManager& manager;

    QLineEdit* txtAptID;
    QComboBox* cmbAptPatientID;
    QLineEdit* txtAptDate;
    QLineEdit* txtAptTime;
    QLineEdit* txtAptDoctor;
    QTableWidget* tableAppointments;
    QPushButton* btnComplete;

    // True when the clinic staff has typed an appointment ID of their own.
    bool aptIdEdited;

    QWidget* buildForm(QWidget* parent);
    QWidget* buildSchedule(QWidget* parent);

    // The next free appointment identifier.
    QString nextAppointmentId() const;

    void refreshPatientCombo();
};

} // namespace gui
} // namespace scapms
```

gui/MainWindow.cpp

```
#include "MainWindow.h"

#include <QAction>
#include <QApplication>
#include <QDateTime>
#include <QFile>
#include <QFileDialog>
#include <QFileInfo>
#include <QGuiApplication>
#include <QIcon>
#include <QKeySequence>
#include <QMenu>
#include <QMenuBar>
#include <QMessageBox>
#include <QScreen>
#include <QShowEvent>
#include <QStatusBar>

#include <memory>
#include <vector>

#include "../utils/ClinicExceptions.h"
#include "../model/GeneralPatient.h"
#include "../model/EmergencyPatient.h"
#include "../model/FollowUpPatient.h"

namespace scapms {
namespace gui {

MainWindow::MainWindow(QWidget* parent) : QMainWindow(parent), centred(false) {
    applyStyleSheet();
    setupUI();
    setupMenuBar();
    setupTimer();

    loadSampleData();
    refreshAll();
}

void MainWindow::showEvent(QShowEvent* event) {
    QMainWindow::showEvent(event);

    // Windows places the window itself, and the screen can be smaller than
    // the window, so the size and the position are only corrected once the
    // window is up.
    if (!centred) {
        centred = true;
        QTimer::singleShot(300, this, [this]() {
            if (QScreen* screen = QGuiApplication::primaryScreen()) {
                const QRect available = screen->availableGeometry();

                // A window that is wider than the screen would hang over the
                // edge, so it is made smaller instead.
                const int margin = 20;
                int wantedWidth = WINDOW_WIDTH;
                if (available.width() > 0 && wantedWidth > available.width() - margin) {
                    wantedWidth = available.width() - margin;
                }
                int wantedHeight = WINDOW_HEIGHT;
                if (available.height() > 0 && wantedHeight > available.height() - margin) {
                    wantedHeight = available.height() - margin;
                }
                resize(wantedWidth, wantedHeight);

                // The layout may still make the window larger than it asked
                // for, so the window is put in the middle a second time with
                // the size it really has.
                QTimer::singleShot(0, this, [this, available]() {
                    move(available.center() - QPoint(width() / 2, height() / 2));
```

gui/MainWindow.h

```
#pragma once

#include <QMainWindow>
#include <QTabWidget>
#include <QTimer>

#include "PatientPanel.h"
#include "AppointmentPanel.h"
#include "QueuePanel.h"
#include "SearchPanel.h"
#include "ReportPanel.h"
#include "AdminPanel.h"
#include "../controller/ClinicManager.h"

namespace scapms {
namespace gui {

// The main window of the clinic application.
//
// It owns the controller and the panels, puts the panels into the tabs and
// keeps them in step with each other. The panels themselves do the work, so
// this class stays small.
class MainWindow : public QMainWindow {
    Q_OBJECT

public:
    explicit MainWindow(QWidget* parent = nullptr);
    ~MainWindow() override;

    // Opens the given tab, counted from zero, and does nothing when the
    // number is out of range. The command line uses this, so that any panel
    // can be opened, photographed or demonstrated directly.
    void showTab(int index);

    // The two panels the command line also fills in, so that the entry point
    // does not have to reach into the widgets of a panel itself.
    QueuePanel* queuePanelWidget() { return queuePanel; }
    SearchPanel* searchPanelWidget() { return searchPanel; }

protected:
    // Used to put the window in the middle of the screen once it is shown.
    void showEvent(QShowEvent* event) override;

private slots:
    // The clock in the status bar.
    void onTimerTick();

    // Called when a panel changed the data.
    void onDataChanged();

    // The undo of the last administrative operation.
    void onUndoAction();

    void onAbout();

private:
    controller::ClinicManager manager;

    QTabWidget* tabWidget;
    QTimer* dashboardTimer;

    // True once the window has been put in the middle of the screen.
    bool centred;

    // The size the window is asked for. The width leaves room for the
    // roster table, whose name and details columns both hold a long text.
    static constexpr int WINDOW_WIDTH = 1200;
    static constexpr int WINDOW_HEIGHT = 660;
```

gui/PatientPanel.cpp

```
#include "PatientPanel.h"

#include <QColor>
#include <QFile>
#include <QFormLayout>
#include <QFont>
#include <QGridLayout>
#include <QGroupBox>
#include <QHBoxLayout>
#include <QHeaderView>
#include <QMessageBox>
#include <QVBoxLayout>

#include <memory>
#include <vector>

#include "../model/Appointable.h"
#include "../model/GeneralPatient.h"
#include "../model/EmergencyPatient.h"
#include "../model/FollowUpPatient.h"
#include "../utils/ClinicExceptions.h"
#include "../utils/DateUtil.h"
#include "../utils/IDGenerator.h"

namespace scapms {
namespace gui {

PatientPanel::PatientPanel(controller::ClinicManager& manager, QWidget* parent)
    : QWidget(parent), manager(manager) {
    QHBoxLayout* layout = new QHBoxLayout(this);
    layout->setContentsMargins(16, 16, 16, 16);
    layout->setSpacing(16);

    layout->addWidget(buildForm(this));
    layout->addWidget(buildRoster(this));

    clearPatientForm();
    setEditingMode(false);
    refresh();
}

QWidget* PatientPanel::buildForm(QWidget* parent) {
    QGroupBox* group = new QGroupBox("Register New Patient", parent);
    QFormLayout* form = new QFormLayout(group);
    form->setContentsMargins(16, 20, 16, 16);
    form->setVerticalSpacing(12);
    form->setLabelAlignment(Qt::AlignLeft | Qt::AlignVCenter);
    form->setFieldGrowthPolicy(QFormLayout::FieldsStayAtSizeHint);

    txtPatID = new QLineEdit(group);
    txtPatID->setPlaceholderText("e.g. GEN-201, EMG-801");
    txtPatID->setToolTip("The identifier of the patient. It has to begin with the prefix "
                         "of the chosen category, and it may only be used once.");

    // The program answers while the identifier is being typed, so that the
    // staff find out straight away that the identifier is already taken.
    connect(txtPatID, &QLineEdit::textChanged, this, &PatientPanel::onPatientIDChanged);

    lblIDHint = new QLabel("The identifier must be unique.", group);
    lblIDHint->setToolTip("Tells you whether the identifier is free or already in use.");
    lblIDHint->setWordWrap(true);
    lblIDHint->setStyleSheet("color: #777; font-size: 11px;");
    // The hint always carries a line, so that the fields below it keep the
    // same place while the staff type.
    lblIDHint->setMinimumHeight(16);

    btnSuggestID = new QPushButton("Suggest ID", group);
    btnSuggestID->setToolTip("Fill in the next free identifier for the chosen category.");
    connect(btnSuggestID, &QPushButton::clicked, this, &PatientPanel::onSuggestPatientID);
```

gui/PatientPanel.h

```
#pragma once

#include <QWidget>
#include <QComboBox>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QTableWidget>
#include <memory>

#include "../controller/ClinicManager.h"
#include "../model/PatientRecord.h"

namespace scapms {

namespace model {
class GeneralPatient;
class EmergencyPatient;
class FollowUpPatient;
}

namespace gui {

// The patient panel. It holds the registration form and the roster table.
//
// The panel owns the form while a record is being edited. When the change is
// saved the snapshot is handed over to the database, which keeps it on the
// undo stack.
class PatientPanel : public QWidget {
    Q_OBJECT

public:
    explicit PatientPanel(controller::ClinicManager& manager, QWidget* parent = nullptr);

    void refresh();

signals:
    void dataChanged();

    // The window owns the undo stack, so the button asks it to undo.
    void undoRequested();

private slots:
    void onPatientTypeChanged(int index);
    void onRegisterPatient();
    void onEditSelectedPatient();
    void onUpdatePatient();
    void onCancelEdit();
    void onDeleteSelectedPatient();
    void onShowReport();
    void onSuggestPatientID();
    void onPatientIDChanged(const QString& text);

private:
    // The order of the entries in the category combo box.
    static constexpr int GENERAL_INDEX = 0;
    static constexpr int EMERGENCY_INDEX = 1;
    static constexpr int FOLLOWUP_INDEX = 2;

    // The two fields that change with the patient category. The panel reads
    // them through the functions below, so it never has to ask what kind of
    // patient it is dealing with.
    struct TypeFields {
        QString label1;
        QString value1;
        QString label2;
        QString value2;
        bool hasSecondField = false;
    };
```

gui/QueuePanel.cpp

```
#include "QueuePanel.h"

#include <QGroupBox>
#include <QHBoxLayout>
#include <QLabel>
#include <QMessageBox>
#include <QVBoxLayout>

#include "../utils/ClinicExceptions.h"

namespace scapms {
namespace gui {

QueuePanel::QueuePanel(controller::ClinicManager& manager, QWidget* parent)
    : QWidget(parent), manager(manager) {
    QHBoxLayout* layout = new QHBoxLayout(this);
    layout->setContentsMargins(16, 16, 16, 16);
    layout->setSpacing(16);

    layout->addWidget(buildControls(this));
    layout->addWidget(buildList(this));

    refresh();
}

QWidget* QueuePanel::buildControls(QWidget* parent) {
    QGroupBox* group = new QGroupBox("Waiting room", parent);
    QVBoxLayout* layout = new QVBoxLayout(group);
    layout->setContentsMargins(16, 20, 16, 16);
    layout->setSpacing(12);

    layout->addWidget(new QLabel("Patient:", group));

    cmbQueuePatientID = new QComboBox(group);
    cmbQueuePatientID->setToolTip("The patient that should join the queue.");
    cmbQueuePatientID->setMaximumWidth(190);
    layout->addWidget(cmbQueuePatientID);

    QPushButton* btnEnqueue = new QPushButton("Add to Queue", group);
    btnEnqueue->setToolTip("Put the patient at the end of the waiting room.");
    connect(btnEnqueue, &QPushButton::clicked, this, &QueuePanel::onEnqueuePatient);
    layout->addWidget(btnEnqueue);

    layout->addSpacing(20);

    QPushButton* btnCallNext = new QPushButton("Call Next", group);
    btnCallNext->setToolTip("Call the patient that has been waiting the longest.");
    connect(btnCallNext, &QPushButton::clicked, this, &QueuePanel::onCallNextPatient);
    layout->addWidget(btnCallNext);

    lblQueueCount = new QLabel("<b>Waiting: 0</b>", group);
    layout->addWidget(lblQueueCount);
    layout->addStretch();

    // The controls keep a minimum width, so that the queue list always has
    // room, and they may grow when the window is wide.
    group->setMinimumWidth(330);
    group->setMaximumWidth(420);
    group->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Preferred);
    return group;
}

QWidget* QueuePanel::buildList(QWidget* parent) {
    QGroupBox* group = new QGroupBox("Queue", parent);
    QVBoxLayout* layout = new QVBoxLayout(group);
    layout->setContentsMargins(16, 20, 16, 16);

    listQueueView = new QListWidget(group);
    listQueueView->setToolTip("The patients that are waiting, the longest wait first.");
```

gui/QueuePanel.h

```
#pragma once

#include <QWidget>
#include <QComboBox>
#include <QLabel>
#include <QLineEdit>
#include <QListWidget>
#include <QPushButton>

#include "../controller/ClinicManager.h"

namespace scapms {
namespace gui {

// The waiting room panel. The queue is first in, first out, so the patient
// that joined first is the one that is called first.
class QueuePanel : public QWidget {
    Q_OBJECT

public:
    explicit QueuePanel(controller::ClinicManager& manager, QWidget* parent = nullptr);

    void refresh();

    // Puts the given patient into the waiting room and redraws the panel.
    // Returns false when that patient is not registered or is already waiting.
    // The command line uses this to open the panel with a patient in it.
    bool enqueueByID(const QString& patientID);

signals:
    void dataChanged();

private slots:
    void onEnqueuePatient();
    void onCallNextPatient();

private:
    controller::ClinicManager& manager;

    QComboBox* cmbQueuePatientID;
    QListWidget* listQueueView;
    QLabel* lblQueueCount;

    QWidget* buildControls(QWidget* parent);
    QWidget* buildList(QWidget* parent);

    void refreshPatientCombo();
};

} // namespace gui
} // namespace scapms
```

gui/ReportPanel.cpp

```
#include "ReportPanel.h"

#include <QComboBox>
#include <QHBoxLayout>
#include <QLabel>
#include <QPushButton>
#include <QTextEdit>
#include <QVBoxLayout>

#include "../controller/ReportBuilder.h"

namespace scapms {
namespace gui {

ReportPanel::ReportPanel(controller::ClinicManager& manager, QWidget* parent)
    : QWidget(parent), manager(manager) {
    QVBoxLayout* layout = new QVBoxLayout(this);
    layout->setContentsMargins(16, 16, 16, 16);
    layout->setSpacing(12);

    QHBoxLayout* topRow = new QHBoxLayout();
    topRow->addWidget(new QLabel("Report:", this));

    cmbReportType = new QComboBox(this);
    cmbReportType->addItem("Patient category distribution");
    cmbReportType->addItem("Patients with pending appointments");
    cmbReportType->addItem("Daily appointment summary");
    cmbReportType->addItem("Frequently attended patients");
    cmbReportType->setToolTip("The report that is built.");
    cmbReportType->setMaximumWidth(340);
    topRow->addWidget(cmbReportType);

    // Choosing another report builds it at once, so that the button does not
    // have to be pressed as well.
    connect(cmbReportType, QOverload<int>::of(&QComboBox::currentIndexChanged),
            this, &ReportPanel::onGenerateReport);

    QPushButton* btnGenerate = new QPushButton("Generate", this);
    btnGenerate->setToolTip("Build the chosen report from the clinic data.");
    connect(btnGenerate, &QPushButton::clicked, this, &ReportPanel::onGenerateReport);
    topRow->addWidget(btnGenerate);

    topRow->addStretch();
    layout->addLayout(topRow);

    txtReportOutput = new QTextEdit(this);
    txtReportOutput->setReadOnly(true);
    txtReportOutput->setFontFamily("Consolas");
    txtReportOutput->setFontPointSize(10);
    layout->addWidget(txtReportOutput, 1);

    onGenerateReport();
}

void ReportPanel::onGenerateReport() {
    const int reportIndex = cmbReportType->currentIndex();

    std::string report;
    if (reportIndex == 0) {
        report = controller::ReportBuilder::categoryDistribution(manager);
    } else if (reportIndex == 1) {
        report = controller::ReportBuilder::pendingAppointments(manager);
    } else if (reportIndex == 2) {
        report = controller::ReportBuilder::dailySummary(manager);
    } else {
        report = controller::ReportBuilder::frequentPatients(manager);
    }

    txtReportOutput->setPlainText(QString::fromStdString(report));
```

gui/ReportPanel.h

```
#pragma once

#include <QWidget>
#include <QComboBox>
#include <QPushButton>
#include <QTextEdit>

#include "../controller/ClinicManager.h"

namespace scapms {
namespace gui {

// The report panel. The reports are built by the report controller, so this
// panel only shows them.
class ReportPanel : public QWidget {
    Q_OBJECT

public:
    explicit ReportPanel(controller::ClinicManager& manager, QWidget* parent = nullptr);

    void refresh();

signals:
    void dataChanged();

private slots:
    void onGenerateReport();

private:
    controller::ClinicManager& manager;

    QComboBox* cmbReportType;
    QTextEdit* txtReportOutput;
};

} // namespace gui
} // namespace scapms
```

gui/SearchPanel.cpp

```
#include "SearchPanel.h"

#include <QFormLayout>
#include <QGroupBox>
#include <QHBoxLayout>
#include <QHeaderView>
#include <QMessageBox>
#include <QVBoxLayout>

#include "../controller/SearchEngine.h"
#include "../controller/SortEngine.h"

namespace scapms {
namespace gui {

SearchPanel::SearchPanel(controller::ClinicManager& manager, QWidget* parent)
    : QWidget(parent), manager(manager) {
    QVBoxLayout* layout = new QVBoxLayout(this);
    layout->setContentsMargins(16, 16, 16, 16);
    layout->setSpacing(14);

    QHBoxLayout* controlRow = new QHBoxLayout();
    controlRow->setSpacing(16);
    controlRow->addWidget(buildSearchBox(this), 3);
    controlRow->addWidget(buildSortBox(this), 2);
    layout->addLayout(controlRow);

    lblAlgorithmExplanation = new QLabel(
        "<b>Selection sort:</b> O(n^2) comparisons, sorts in place.<br>"
        "<b>Merge sort:</b> O(n log n) comparisons, needs an extra array.", this);
    lblAlgorithmExplanation->setWordWrap(true);
    layout->addWidget(lblAlgorithmExplanation);

    tableSearchResults = new QTableWidget(this);
    tableSearchResults->setColumnCount(5);
    tableSearchResults->setHorizontalHeaderLabels(
        {"ID", "Full Name", "Age", "Phone", "Category"});

    // The identifier, the name, the age and the telephone number take the
    // width of their content, so that none of them is cut in half. The
    // category column takes the rest of the table.
    QHeaderView* header = tableSearchResults->horizontalHeader();
    header->setSectionResizeMode(QHeaderView::Interactive);
    header->setSectionResizeMode(0, QHeaderView::ResizeToContents);
    header->setSectionResizeMode(1, QHeaderView::ResizeToContents);
    header->setSectionResizeMode(2, QHeaderView::ResizeToContents);
    header->setSectionResizeMode(3, QHeaderView::ResizeToContents);
    header->setSectionResizeMode(4, QHeaderView::Stretch);
    header->setMinimumSectionSize(45);
    tableSearchResults->verticalHeader()->setVisible(false);
    tableSearchResults->setSelectionBehavior(QAbstractItemView::SelectRows);
    tableSearchResults->setEditTriggers(QAbstractItemView::NoEditTriggers);
    tableSearchResults->setTextElideMode(Qt::ElideRight);
    layout->addWidget(tableSearchResults, 1);

    onSearchAlgorithmChanged(SEARCH_LINEAR_ID);
    onSortAlgorithmChanged(0);

    // No search has been run yet, so there is nothing to repeat.
    lastQuery.clear();
    lastAlgorithm = SEARCH_LINEAR_ID;
}

QWidget* SearchPanel::buildSearchBox(QWidget* parent) {
    QGroupBox* group = new QGroupBox("Search", parent);
    QFormLayout* form = new QFormLayout(group);
    form->setContentsMargins(16, 20, 16, 16);
    form->setVerticalSpacing(10);
```

gui/SearchPanel.h

```
#pragma once

#include <QWidget>
#include <QComboBox>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QTableWidget>

#include "../controller/ClinicManager.h"
#include "../controller/SortEngine.h"

namespace scapms {
namespace gui {

// The search and sort panel.
//
// The user chooses the algorithm, so the program shows which routine was used
// and how the records were ordered.
class SearchPanel : public QWidget {
    Q_OBJECT

public:
    explicit SearchPanel(controller::ClinicManager& manager, QWidget* parent = nullptr);

    void refresh();

    // Runs the given search without showing a message box, and redraws the
    // results table. Returns the number of records that matched. The command
    // line uses this to open the panel with a search already shown.
    int runQuery(const QString& query, int algorithm);

signals:
    void dataChanged();

private slots:
    void onExecuteSearch();
    void onExecuteSort();
    void onSearchAlgorithmChanged(int index);
    void onSortAlgorithmChanged(int index);

private:
    controller::ClinicManager& manager;

    QLineEdit* txtSearchQuery;
    QComboBox* cmbSearchAlgorithm;
    QLabel* lblSearchCategory;
    QComboBox* cmbSearchCategory;
    QComboBox* cmbSortAlgorithm;
    QComboBox* cmbSortField;

    QTableWidget* tableSearchResults;
    QLabel* lblAlgorithmExplanation;

    // The last search that was run, so that it can be run again when the
    // data changed. An empty text means that no search was run yet.
    QString lastQuery;
    int lastAlgorithm;

    // The entries of the two algorithm combo boxes.
    //
    // The linear search has two entries, because the brief asks for a linear
    // search by patient ID, and a keyword search across the identifier and
    // the name is what the staff actually type. Both are linear scans, but
    // they compare different fields, so they are kept apart.
    enum SearchAlgorithm {
        SEARCH_LINEAR_ID = 0,
        SEARCH_LINEAR_KEYWORD = 1,
        SEARCH_BINARY_ID = 2,
```

gui/clinic.ico

```
□        □□□ [□□□□□ j□□□□□□ □□[□  □ □□□□((□ □□□00□ f□□□@@□ □□!□□□□ J□□
□ □
*□□PNG
□

IHDR□□□□□□□a□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□IDAT8Oc□:0□□7□□□□□□□□□C□□□□□~□□□□□@□□      a□□□□□=□P□□`□ᒪ□□b<
□□}|=□X□Y|□□□□□□□□gm8□□  6□<□□□□□□
0□□/@□□i□□□□]□□□□□□□0A□□go<□j□□f□\□□H.□4□□□□□□□□{□□A
a□F□□b□□□}□□$□□□.I□□□□
6□□L□□□#K□□□□□□□□□□□□4□K□l□□R#P□9□Bl□□□$□□7□□□□□
□*A□□□□□□630□□□f□%□CIEND□B`□□PNG
□

IHDR□□□□□□□□
□sRGB□□□□□gAMA□□□□a□     pHYs□□□□□□□o□d□IDAT8Oᑭ!□□`□□□F□?□□□□lV□□vЬF□T□□u□□□f4□□□□□□□□□□@□ᵢM7}□g□□□ℭ□u9□
□□□□)□□□□Y□□C□i{j□□q]□\ʯu□D%r□□  □kp□T□a.□}□□L□□x□□2PK>□□□q□Mj&□B□B-□>□□□□□□□□Y竈J□:□      □|□□□□□M
□□□□□□□□`^□%□□[□□¨&S\□□7□□Ph□G□3□ë□□□"□*□O□ᵩU□□!\□□□M2□□□0□□□□z□□lry□*□□□□,□□□□D|X)□6IEND
□B`□□PNG
□

IHDR□□□□□w=□□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□"IDATHKc□□`□□□o□    □□□
Dc□z□□^□□□□H□P□u  □O.6I□□n□□□□5□□□□c□D□□□□j4□□}|=□Ib□}□
□□}□M□$□□P□□Pl□□□□□□$□□□□d□ꝩ□□□
□Q0H□Z$□□□□□□□ □□□H□Z$<□-@□□7□C□E□□□,q□□□□□R□H□□9□□□□□,□y$□c□□□$□M-□□□□□}□X$Q0□□□z`□D□□q□□X
□□□□□e□-Ꮼ□□□D□*□□□□h□-□□m□□□□□□□5□□@qA
□k□□□Z□j,*0□□W□V@□□E□□□%□`□□j□□00□-□;□]□iIEND□B`□□PNG
□

IHDR  □□szz□□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□\IDATXGc□□□8□G□4□?□4□□□l□□□o5□0□□□□0I□o7I□□□4□?□□r□□.
□U□□R□□□□h□□□□□□{n□□□□□□□>O갣M□□□q□{□□□l:vŴ□@□□□C□□□□&q□□□□□+□□□{□□□□□□□gm8□□A□□□□□f□□□`□□□□□`S□□□,□□@□□□□□
`□□□h□x□□F□□p□□□7□C□D□ ql□□□□□□□i□□□□□□| □u□□□□;`□s□.2□□□cS□□□:`□8 □□□□$NLu□□Z□X$qb\□□Ojă□}□□□□□□□□□*□5F□□□□□*□□□u□□VC□□Fϖ1
□HI|o□=□□j□C□□□A8□M#□□h.□n□6`□□k□□@□□2□"□1v□□□□H□□SA□rIEND□B`□□PNG
□

IHDR((□□□□□m□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□□IDATXG뼩O□@□□+□H$□□□□9¿□□=□□D□#□□□1H□□ă0E□□!□□I□_□`l}
□□□z□□~ɗ,□{w_□□[zẅ□.□f□□ !>NR□□3□□□□□□L□□$□□□4□6□M<0□□Lm□6□□□U□i□8□Ŵ□6D□□4□□W□□□□□r□M□□r□□□J□Ei□□□#□skP~&jϋ□□
□□cQ□"□⅃□□□.□□□□□□□□□aX      1□□□□□:ᒍ□□□□H⸗v□AA!□□□
□□□ □□
□□□□□MT□□]□□□G□□  □k"□W□□□AK X7A□nV&□□□A□□□□□ku□□□  □□?□□□Tit□`□□v□□□□□j(□□□]□□□□mC□S□"□□W;□
D□□□>□□□^□□□       p□(j□□□ㅛ□□z□□□Y□□XZ□□+□□D□□yaF□"□`□r_)□□□/□A□□F□□<□x□\□K□Z□□□Θ{[□□E□□□□,%□N□IEND□B`
□□PNG
□

IHDR00□□W□□□□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□□IDAThC횡O□1□□'□H$□□□□□□□$□W65□□cn□pWI□A"□□□□□□$□  □□□□□□□`□□□¿□□
□_□K.□□□□□□kr]mcT□7{u!Ob!□□□Ⴆ□9□]/□}!□□□□G□tD□ǯ□Z□□□□□□□.□\□H□H□%vO□□□ϵ□K□:\□□□□□□e□l□
Z|□□□□Rg□e□5□□□vu□□ↄ□/k□□□□oE]z&□□□□□□Z□O□□L□2□9□□□□U<6
.Ü□□□□□y□□□q□mi□y□□□□□□ř□q\{["!/□□\p□□□6@□□□□\L□X□`□□`□@0`□□□0ᵋ^□?t□□B□□>□b□37□Ⴆ□□□□□□□
(□□□□,□p□□□Z□□□Ϥ M@□&B□□~      □□□□□=□@□□□□□□□2□□H0□□□□□=_N□□]□□q□□□□W□□□□□□□rA>s(□2□{;□□+T□P□
□       □□□i□~□Ⴗ□p□N□□K.*{□□!□□□□IR□Q+□□(□m□□□□□;i□R□K□D[□□$□@#□□□{Z
□□Q□□□□□□□G□□3□'□oi□□PB<□N□□□□Y□□j□□□□Vn□□□□IEND□B`□□PNG
□

IHDR@@□□□iq□□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□TIDATx^□/O□1□□'□H$□□□n□□□a□W65□□1G □□1□   □8>□□z8□□9□□}ɔ□□w□□R
□□O□d□□□□□□□□%mCTA-□□{J□□FVW□□ ₚ□K&51□□□□□;”□□□□bv□□□[*□□*□I!T□*2A□q7□□□4S□□□H□□□□|ŵ□d_□m
m□□□<F/LE5v>□8A!.□□□ₔ4 \□P□?□w%□Ã5:w□□□□+□□{□□□□D□_□□□□v*WW[□□□;.s□□z□□Bw2T□□□    {B□□□
□□Wx□=□□□jx□eH□m□;□@w□!2□□□>□□□ᵾ\]□□Y□□□□□□□
e□□□z`□□□□□@□
□□□□□l]□-□□□□□wm□□□Ja□□□puq□□□P□n□□□□□□+,□□□□@□□□□@□□u□□□
`□?□|□F□
e□□□Z□□  □□□E!!□□@□T~□$□u□25<f□□□w□W\ᓚ□□x□□□□□2$nt□□□m□\□G□□□□.□/□Bw2 □□L□□□n~K□;□5X□□□□    3□g3'f
□*□□□□□□□□□□)_zl□#cᏞ□|□[&dJ□□[□□□[□□S□□□□□Y□猛□□Xa□□□o□;_F□20o□g□□O□□AI_□Aκ/□□)*□]~A7□□□y□
□□□
□IEND□B`□□PNG
□

IHDR□□□□□>a□□sRGB□□□□□gAMA□□□□a□          pHYs□□□□□□o□d□□IDATx^□/p□7□□
□□□□Ò□□%e       ,,S□Ô□,f□f□□□L□ɻ□□□□u□SfP`□□□□□=□□.o/:kwOZ}□□□7□w□□z?i□□□#□ 9v□□□p□□b□Ⅹ7□□m□h□□□□□"□□e
{□□~□8□□□□□□=]□□□□I□GN□q□□Ꮼ□□□1□□!□□¨c□□6u^HooN□□f□n□<□□*□QA{□□□Dq$□□I□□□□e□□□□□'□□5□□
q□/□$□!□□□□<i□□f\□□□□B□□'□□U□□F□$t□□□□□□□□□□□□□dá=□□□□□□□□  □□K□□□N□□Kc}□g□□□v□□□<□f□_□□nMK
```

gui/resources.qrc

```
<RCC>
    <qresource prefix="/">
        <file>style.qss</file>
        <file>clinic.ico</file>
    </qresource>
</RCC>
```

gui/style.qss

```
/* The look and feel of SCAPMS. One flat palette for every panel. */

QMainWindow {
    background-color: #f7f8fa;
    font-family: 'Segoe UI', Arial, sans-serif;
}

QTabWidget::pane {
    border: 1px solid #dde1e7;
    background: #ffffff;
    border-radius: 8px;
    margin-top: -1px;
}

QTabBar::tab {
    background: #e9edf2;
    color: #5b636e;
    font-weight: 600;
    padding: 10px 20px;
    border-top-left-radius: 6px;
    border-top-right-radius: 6px;
    margin-right: 4px;
}

QTabBar::tab:selected {
    background: #ffffff;
    color: #35618f;
    border-top: 3px solid #35618f;
    border-bottom: 1px solid #ffffff;
}

QTabBar::tab:hover:!selected {
    background: #dde1e7;
    color: #25292f;
}

QGroupBox {
    font-weight: 600;
    color: #25292f;
    border: 1px solid #dde1e7;
    border-radius: 8px;
    margin-top: 18px;
    padding-top: 14px;
    background: #ffffff;
}

QGroupBox::title {
    subcontrol-origin: margin;
    subcontrol-position: top left;
    left: 14px;
    padding: 0 6px;
    color: #35618f;
}

QLineEdit, QComboBox {
    background: #ffffff;
    border: 1px solid #c4ccd6;
    border-radius: 6px;
    padding: 7px 10px;
    font-size: 13px;
    color: #25292f;
    selection-background-color: #35618f;
}

QLineEdit:focus, QComboBox:focus {
    border: 1.5px solid #35618f;
    background: #f7f8fa;
}
```

gui_main.cpp

```
// COS202 Programming II - Lab Assessment
// Smart Clinic Appointment & Patient Management System (SCAPMS)

#include <QApplication>
#include <QMessageBox>

#include <exception>
#include <string>

#include "gui/MainWindow.h"

namespace {

// The application of the clinic.
//
// The class exists for one reason: Qt does not catch an exception that
// leaves a slot. Without this class an error that slips past the checks of
// a panel would travel out of the event loop and end the program with
// "terminate called after throwing an instance of ...". The clinic has to
// keep running after a normal mistake, so the exception is caught here and
// shown to the staff as a message.
class ClinicApplication : public QApplication {
public:
    ClinicApplication(int& argc, char** argv) : QApplication(argc, argv) {}

protected:
    // Every event of the program passes through this method, so it is the
    // one place where an exception can still be stopped.
    bool notify(QObject* receiver, QEvent* event) override {
        try {
            return QApplication::notify(receiver, event);
        } catch (const std::exception& error) {
            QMessageBox::critical(nullptr, "Unexpected error", error.what());
            return false;
        } catch (...) {
            QMessageBox::critical(nullptr, "Unexpected error",
                                  "An unknown error stopped this action.");
            return false;
        }
    }
};

} // namespace

int main(int argc, char* argv[]) {
    ClinicApplication app(argc, argv);

    scapms::gui::MainWindow mainWindow;
    mainWindow.show();

    // The panels can be asked for on the command line:
    //
    //   SCAPMS_GUI.exe                      opens the first panel
    //   SCAPMS_GUI.exe 3                    opens the Waiting Queue
    //   SCAPMS_GUI.exe 3 GEN-101            also puts that patient in the queue
    //   SCAPMS_GUI.exe 2 GEN-101 0          also runs a search for GEN-101
    //                                      with the linear search by ID
    //
    // The number is the position of the tab, in the order the tabs are shown.
    // The second and third words only mean something for the two panels they
    // belong to, so an argument that does not fit the panel is ignored. This
    // is how the screenshots of the submission were produced, and it makes a
    // demonstration of any single panel possible without clicking.
    if (argc > 1) {
        bool numberOk = false;
        const int requested = QString::fromLocal8Bit(argv[1]).toInt(&numberOk);
        if (numberOk && requested >= 0 && requested <= 4) {
            mainWindow.showTab(requested);
```

4.4 Data files required to run the application

appointments.csv

```
APT-1001,GEN-101,2026-11-15,10:30 AM,Dr. Adeyemi,Pending
APT-1002,FOL-205,2026-11-20,02:00 PM,Dr. Funke,Completed
APT-1003,GEN-005,2026-11-22,11:00 AM,Dr. Adeyemi,Pending
```

patients.csv

```
General,GEN-101,Amaka Okonkwo,28,+2348073451298,4500.00
Emergency,EMG-999,Daniel Ojukwu,22,+2348034874562,1,Acute trauma
FollowUp,FOL-205,Babatunde Alabi,45,+2348067624513,2026-10-20,Dr. Funke
General,GEN-005,Chioma Eze,19,+2348091837465,5000.00
Emergency,EMG-102,Zainab Bello,32,+2348019283756,2,Fracture
FollowUp,FOL-206,Ngozi Emeka,61,+2348075192648,2026-11-02,Dr. Okafor
```

5. Readme and Supplementary Report

5.1 Readme

Smart Clinic Appointment & Patient Management System (SCAPMS)

Course: COS202 Programming II - Lab Assessment Project
Author: OlisaEmeka Davies Chinedu-Eze
Technologies: C++17, Qt 6.5.3 (x64), MSVC 2022

Project Overview

A desktop application for running an outpatient clinic. Patient records, appointments and the waiting room are all managed from a Qt widgets interface, backed by `std::vector`, `std::queue` and `std::stack`. The search and sort algorithms are written by hand rather than taken from the standard library.

The program is split into four layers, and every layer has its own namespace:

Layer	Folder	Namespace	What it holds
Model	`model/`	`scapms::model`	The patient classes, the appointment, the database
Controller	`controller/`	`scapms::controller`	The searching, sorting and reporting routines
GUI	`gui/`	`scapms::gui`	The main window and one panel per tab
Utilities	`utils/`	`scapms::utils`	The exception classes, the file reader, the ID generator

The model, controller and utility layers do not use Qt at all, so the same code runs in the console demonstration (`main.cpp`) and in the graphical program (`gui_main.cpp`).

Features

- Patient records for the general, emergency and follow-up categories, with add, edit and delete
- Appointment scheduling and cancellation
- Waiting room queue, served first in first out
- Undo for the last change to a patient or an appointment
- Patient search by linear, binary, recursive and category search, and sorting by selection sort and merge sort
- CSV import and export for patients and appointments
- Reports on category distribution, pending appointments, the daily schedule and frequently attended patients

Folder layout

```
model/       PatientRecord, Appointable, Billable, ScheduledPatient, GeneralPatient,
             EmergencyPatient, FollowUpPatient, Appointment, AdminAction, ClinicDatabase
controller/  ClinicManager, AppointmentController, SearchEngine, SortEngine, ReportBuilder
```

```
gui/         MainWindow, PatientPanel, AppointmentPanel, QueuePanel,
             SearchPanel, ReportPanel, AdminPanel, style.qss, clinic.ico,
             resources.qrc
utils/       ClinicExceptions, DateUtil, FileHandler, IDGenerator, StringUtil
main.cpp     the console demonstration
gui_main.cpp the entry point of the graphical program
```

The source code under the headings of the submission

The submission asks for the complete source code under four headings. The files of the project belong to them like this:

i. Header files

```
model/PatientRecord.h          model/Appointable.h        model/Billable.h
model/ScheduledPatient.h       model/GeneralPatient.h      model/EmergencyPatient.h
model/FollowUpPatient.h        model/Appointment.h         model/AdminAction.h
model/ClinicDatabase.h         controller/ClinicManager.h  controller/AppointmentController.h
controller/SearchEngine.h      controller/SortEngine.h     controller/ReportBuilder.h
utils/ClinicExceptions.h       utils/DateUtil.h            utils/FileHandler.h
utils/IDGenerator.h            utils/StringUtil.h          gui/MainWindow.h
gui/PatientPanel.h             gui/AppointmentPanel.h      gui/QueuePanel.h
gui/SearchPanel.h              gui/ReportPanel.h           gui/AdminPanel.h
```

ii. Source files

```
model/PatientRecord.cpp        model/ScheduledPatient.cpp  model/GeneralPatient.cpp
model/EmergencyPatient.cpp     model/FollowUpPatient.cpp   model/Appointment.cpp
model/AdminAction.cpp          model/ClinicDatabase.cpp    controller/ClinicManager.cpp
controller/AppointmentController.cpp                       controller/SearchEngine.cpp
controller/SortEngine.cpp      controller/ReportBuilder.cpp utils/DateUtil.cpp
utils/FileHandler.cpp          utils/IDGenerator.cpp       main.cpp
```

iii. GUI files and resources

```
gui/MainWindow.cpp              gui/PatientPanel.cpp        gui/AppointmentPanel.cpp
gui/QueuePanel.cpp              gui/SearchPanel.cpp         gui/ReportPanel.cpp
gui/AdminPanel.cpp              gui_main.cpp                gui/style.qss
gui/clinic.ico                  gui/resources.qrc           SCAPMS.pro
build_gui.bat                   build.bat                  .vscode/tasks.json
.vscode/launch.json
```

iv. Data files

```
patients.csv                    appointments.csv
```

`patients.csv` and `appointments.csv` hold the records that the import and the export dialogs read and write. The program starts with the same records built into it, so it also runs without them.

The program has two entry points: `gui_main.cpp` builds the graphical program and `main.cpp` builds the console demonstration. They cannot be in one binary because each of them has its own `main()`, so the console version is built by `build.bat` with the compiler directly and is not part of `SCAPMS.pro`.

Opening a panel from the command line

The panels of the graphical program can also be asked for on the command line, which is how the screenshots in the submission were produced and which makes a demonstration of any single panel possible without clicking:

```
SCAPMS_GUI.exe                    opens the first panel
SCAPMS_GUI.exe 3                  opens the Waiting Queue
SCAPMS_GUI.exe 3 GEN-101          also puts that patient in the queue
SCAPMS_GUI.exe 2 GEN-101 0        also runs a search for GEN-101 with the
                                  linear search by ID
```

The number is the position of the tab, in the order the tabs are shown: 0 Patient Records, 1 Appointments, 2 Waiting Queue, 3 Search & Sort, 4 Clinic Reports, 5 Administration. The second and the third words only mean something for

the two panels they belong to, so an argument that does not fit the panel is ignored.

The submission document

`SCAPMS_Submission.pdf` is the single document the assessment asks for. It holds, in the order the brief lists them: the two to three page report, the six screenshots, the UML class diagrams, and the complete source code under the four headings the brief names. `SUBMISSION_REPORT.md` is the report on its own, and the four `UML_*.png` pictures are the diagrams, because a full diagram of Twenty five classes do not read on one page when they are printed.

Where each requirement is implemented

Requirement	Implementation
1.a Abstract base class	`model/PatientRecord.h` - `getTypeLabel()`, `getPatientReport()`, `getSpecialDetail()` and `clone()` are pure virtual; `displayPatient()` and `restoreFrom()` are concrete methods shared by every category
1.b Subclasses	`model/GeneralPatient.h`, `model/EmergencyPatient.h`, `model/FollowUpPatient.h`
1.c Interface	`model/Appointable.h` - `scheduleAppointment()`, `cancelAppointment()` and the overloaded `cancelAppointment(id)`; `model/Billable.h` is a second interface, so a class can implement more than one
1.d Runtime polymorphism	`main.cpp` walks the records through a `PatientRecord*` and calls `displayPatient()`, which dispatches to the version of the category; `gui/PatientPanel.cpp` shows the same report through the base pointer
1.e Composition	`model/PatientRecord.h` owns the `std::vector<Appointment> appointments` collection, so a patient holds one or more appointments; `model/ClinicDatabase.h` owns the patient records
1.f / 2 Organisation	The four layers above, each with its own namespace; every class with behaviour is split into a header and a source file
2 Separation of responsibilities	`controller/ClinicManager.h` is the only class the GUI talks to. `controller/AppointmentController.h` holds the rules of a booking. `model/ClinicDatabase.h` only stores and finds. The GUI never calls the database directly.
3.a Vector	`model/PatientRecord.h` - `std::vector<Appointment> appointments`; `model/ClinicDatabase.h` - `std::vector<std::unique_ptr<PatientRecord>> patients`
3.b Queue	`model/ClinicDatabase.h` - `std::queue<PatientRecord*> waitingQueue`. The patient who joins first is served first
3.c Stack	`model/ClinicDatabase.h` - `std::stack<AdminAction> undoStack`, with `model/AdminAction.h` holding the copy of the record that is put back. Undoing a delete also gives the record its place in the waiting queue again
3.d Fixed-size array	`model/ClinicDatabase.h` - `PatientRecord* recentPatients[10]`, shown in the Administration panel. The array is held from the oldest record at the front to the newest at the back, the most recently used record is shown first, and when the cache is full the record that was used the longest time ago makes room for a new one
4 Searching	`controller/SearchEngine.h` - linear by ID or name, binary by ID on sorted records, recursive by name, linear by category. The text is matched without regard to letter case, and a part of the identifier or the name is enough
5 Sorting	`controller/SortEngine.h` - selection sort and merge sort, chosen from a combo box on the Search & Sort panel. The chosen order is kept by the controller, so the roster and the results show the same order. The names and the identifiers are compared without regard to letter case
6 Events	`gui/MainWindow.cpp` and the panels - button clicks, menu actions, combo box changes, text field changes and a `QTimer` that refreshes the status bar and the dashboard. The File menu holds New, Save as, Load and Exit
7 Panels	Patient Records, Appointments, Waiting Queue, Search & Sort, Clinic Reports, Administration
8 Layout	`QFormLayout` for the forms, `QHBoxLayout` and `QVBoxLayout` for the panels, `QGridLayout` for the tables of the categories and their priorities

Requirement	Implementation
9 Advanced GUI	9.a colour coded rows with a table of the categories and their priorities, 9.b fields that change with the patient type, 9.c file dialogs, 9.d status bar timer, 9.e validation messages, 9.f shortcuts, 9.g tooltips
10 Exceptions	`utils/ClinicExceptions.h` - `DuplicateIDException`, `InvalidPatientDataException`, `InvalidAppointmentException`, `FileIOException`. `gui_main.cpp` also catches anything that escapes a slot, so that the program never stops by itself
11 Persistence	`utils/FileHandler.h` - CSV read and write for both patients and appointments. A file is written to a temporary name first, so a failed save cannot destroy the data that is already on the disk

The rules of the clinic

The rules live in the controller layer, so that the screens and the data always agree on them.

Rule	Where it is checked
The age has to be between 0 and 120	`ClinicManager::checkPatientRules`
The name may not be empty	`ClinicManager::checkPatientRules`
The identifier has to begin with the prefix of the category: `GEN-`, `EMG-` or `FOL-`	`ClinicManager::checkPatientRules`
A contact number that is given has to look like a telephone number	`ClinicManager::checkPatientRules`
The same patient may not wait in the queue twice	`ClinicManager::enqueuePatient`
The date has to be written as `YYYY-MM-DD` and the time as `HH:MM AM` or `HH:MM PM`	`AppointmentController::checkBookingRules`
The same doctor cannot be booked twice in the same slot	`AppointmentController::scheduleAppointment`
An emergency patient is not booked in advance, it goes to the waiting room	`AppointmentController::scheduleAppointment`
A visit that was completed cannot be cancelled, and a cancelled visit cannot be completed	`ScheduledPatient::cancelAppointment`

Screenshots

The `screenshots/` folder holds the six views of the graphical program, in the order in which the submission asks for them:

File	View
`1_main_dashboard.png`	The dashboard of the clinic, with the live figures and the categories
`2_patient_registration.png`	The registration form and the roster of the patients
`3_appointment_management.png`	The booking form and the schedule of the clinic
`4_search_and_sort.png`	The search and the sort, with the result of a search
`5_waiting_queue.png`	The waiting room and the patients that are waiting
`6_clinic_reports.png`	The report of the clinic

Build & Run

Prerequisites

- Windows 10/11 (x64)
- Visual Studio 2022 C++ Build Tools
- Qt 6.5+

Compilation

Run the build script for the graphical program:

```
build_gui.bat
```

The binary is generated at `release\SCAPMS_GUI.exe`.

`build.bat` builds the console version (`build\SCAPMS.exe`), which walks through the exception handling, the polymorphic calls, the searching and sorting routines, the waiting room, the undo stack, the summary of the clinic and the four reports without needing the GUI.

Data files

`patients.csv` and `appointments.csv` hold the same records the application starts with, and are the files the import and export dialogs read and write. The program starts with the same records built into it, so it also runs when these two files are not there.

5.2 Detailed supplementary report

The report above follows the twelve items the brief lists. This fuller account keeps the record of every problem met while the program was built, and is included as supporting material.

Academic Project Report

Course: COS202 Programming II
Project Title: Smart Clinic Appointment & Patient Management System (SCAPMS)
Author / Student: OlisaEmeka Davies Chinedu-Eze
Language & Framework: C++17, Qt 6.5.3 (x64)
IDE & Build Environment: Visual Studio 2022 MSVC Build Tools / Visual Studio Code

---

1. Project Overview & Problem Statement

A clinic still running on paper or a spreadsheet ends up with double bookings, a waiting room nobody can keep track of, deleted records with no way back, and no way to see how the day is going. This project fixes those problems with a desktop application.

The **Smart Clinic Appointment & Patient Management System (SCAPMS)** is a desktop application for running an outpatient clinic. It handles: 1. Patient demographic registration and categorization. 2. Advance consultation scheduling and status tracking. 3. First-In, First-Out (FIFO) consultation waiting queue coordination. 4. Fast manual searching and multi-attribute record sorting. 5. Administrative transaction history with instant rollback (Undo). 6. File persistence via structured Comma-Separated Values (CSV). 7. Diagnostic reporting of clinic attendance trends.

2. Object-Oriented System Architecture

The code is split into four layers - Model, Controller, GUI and Utilities - and each layer has its own C++ namespace (`scapms::model`, `scapms::controller`, `scapms::gui`, `scapms::utils`). Every class that has behaviour is written as a header file with the declarations and a source file with the implementation; the two pure interfaces and the small inline helper headers are header only by design. The Model, Controller and Utility layers do not depend on Qt, so the same code also runs in the console demonstration.

2.0 Separation of Responsibilities (Requirement 2)

The program is built as three layers that only ever depend on the layer below them:

```
GUI layer          PatientPanel, AppointmentPanel, QueuePanel,
   |                SearchPanel, ReportPanel, AdminPanel, MainWindow
   |  asks the controller to do the work
   v
Controller layer    ClinicManager, AppointmentController,
   |                SearchEngine, SortEngine, ReportBuilder
   |  applies the rules of the clinic, then stores the result
   v
Model layer         PatientRecord and its subclasses, Appointment,
                    AdminAction, ClinicDatabase
```

The rule that makes the separation real is that **the screens never call the database directly**. Every panel holds a `ClinicManager&` and asks it to do the work, in the same way that the console demonstration does. Before this restructure the panels called `ClinicDatabase` themselves, which meant the business rules and the data handling had ended up in one class of 488 lines. The rules now live in the controller layer:

- `ClinicManager` is the single point of contact for the GUI. It owns the database and the appointment controller, and it applies the rules of a patient record (the age has to be between 0 and 120, the name may not be empty, the identifier may only be used once).
- `AppointmentController` holds the rules of a booking (the date and the time have to be written in the format of the clinic, the patient has to be registered, and the category of the patient has to be one that can be booked in advance).
- `ClinicDatabase` now only stores and finds. The few exceptions it still raises are the ones that follow from the shape of the data itself, such as an identifier that is already in use.

The classes named in the brief are all present in their own layer: `ClinicManager`, `SearchEngine` and `AppointmentController` in the Controller, `MainWindow`, `PatientPanel`, `AppointmentPanel`, `AdminPanel` and `SearchPanel` in the GUI, `IDGenerator` and `FileHandler` in the Utilities, and the patient classes and `Appointment` in the Model.

The `AdminPanel` is a sixth tab that holds the operations that belong to the person in charge of the clinic rather than to the treatment of a patient: undoing the last step, saving and loading the data, and starting a new day with an empty database. It also shows the live dashboard of the clinic and the records that were used most recently, which are the ten records held in the fixed size cache. The dashboard is built out of real widgets rather than out of one block of text: a table of the six figures of the clinic, and a table of the three categories with the priority that goes with each of them.

2.1 Abstract Base Class & Inheritance Hierarchy

At the core of the clinical model is the abstract base class `PatientRecord` (`model/PatientRecord.h`): - **Encapsulated Attributes:** `patientID`, `name`, `age`, `contactNumber` and `patientType`. - **Pure Virtual Functions (`= 0`):** `getTypeLabel()`, `getPatientReport()`, `getSpecialDetail()` and `clone()`. Each derived class has to provide its own version of them. - **Concrete Methods:** `displayPatient()` and `restoreFrom()` are implemented in the base class and shared by every category. `displayPatient()` writes the report that the category produced, so it is also the method that the console demonstration calls through a base-class pointer. - **Virtual Destructor:** Ensures safe polymorphic deallocation through base pointers without resource leakage.

Three specialized subclasses inherit from `PatientRecord`: 1. `GeneralPatient`: Represents standard outpatient visits. Encapsulates a `consultationFee` attribute, which is quoted in naira. 2. `EmergencyPatient`: Models urgent trauma admissions. Encapsulates an urgency `severityLevel` (1-3) and a qualitative `triageNote`. 3. `FollowUpPatient`: Facilitates continuity of care. Encapsulates a `previousVisitDate` and `assignedDoctor`.

2.2 Interface Design (`Appointable` and `Billable`)

C++ does not possess a native `interface` keyword. Consequently, abstract interfaces were constructed as pure abstract classes possessing solely pure virtual functions and a virtual destructor:

```
class Appointable {
public:
    virtual ~Appointable() = default;
    virtual bool scheduleAppointment(const model::Appointment& appointment) = 0;
    virtual bool cancelAppointment() = 0;
    virtual bool cancelAppointment(const std::string& appointmentID) = 0;
    virtual bool hasScheduledAppointment() const = 0;
    virtual std::string getAppointmentDetails() const = 0;
};
```

`Appointable` also demonstrates **compile-time polymorphism** through method overloading: the two `cancelAppointment()` methods share a name and differ only in their parameter list, so the choice between cancelling the next booking and cancelling a named one is made by the compiler.

A second interface, `Billable` (`model/Billable.h`), declares `getBillAmount()` and `describeBill()`. The intermediate abstract class `ScheduledPatient` inherits from the base class **and from both interfaces at the same time**, which demonstrates that a class may implement more than one interface:

```
class ScheduledPatient : public PatientRecord, public Appointable, public Billable { ... };
```

`GeneralPatient` and `FollowUpPatient` therefore answer both interfaces, while `GeneralPatient` overrides the billing methods with its consultation fee. The shared booking code is written once in `ScheduledPatient`, so the two subclasses only add their own category fields.

By clinical logic, `EmergencyPatient` deliberately omits `Appointable`, reflecting hospital protocol where trauma cases cannot be scheduled in advance. Because the database routes every booking through the `Appointable` interface, this rule is enforced by the type system rather than by a check on the category name: an attempt to book an emergency case is rejected with a `InvalidAppointmentException`, and the emergency case reaches the doctor through the waiting room instead.

2.3 Polymorphism & Composition

- **Runtime Polymorphism:** All patient records are managed through base-class pointers (`PatientRecord*`). Virtual function dispatch dynamically routes calls to derived subclass implementations via internal virtual method tables (VTables). The console demonstration calls `displayPatient()` through a `PatientRecord*`, and the graphical panel builds the same report through `getPatientReport()`, so the polymorphic call is exercised by both programs.
- **Composition ("HAS-A"):** A patient owns its appointments. `PatientRecord` holds a `std::vector<Appointment> appointments` collection, so a patient is associated with one or more appointments and the appointments are created and destroyed together with their owner. The `ClinicDatabase` owns the patient records in a `std::vector<std::unique_ptr<PatientRecord>>` and gathers the appointments of all patients on demand for the schedule table and the reports.
- **Ownership:** The database owns the live records and deletes them in its destructor. The undo stack owns one *copy* of the record for each administrative action, so the stack and the roster never share the same memory. The undo entry holds the copy in a `std::unique_ptr`, which makes it movable but not copyable, and frees the copy automatically when the entry is destroyed.

3. Data Structures & Justification

Data Structure	C++ Implementation	Complexity	Clinical Justification in SCAPMS
Vector	`std::vector<std::unique_ptr<PatientRecord>>`	O(1) amortized append	Manages the dynamic list of registered patients. The unique pointers keep the ownership of each record explicit, so a record is freed exactly once.
Vector	`std::vector<Appointment>`	O(1) amortized append	The appointments a patient holds. The patient owns them, so they are created and destroyed together with the record.
Queue	`std::queue<PatientRecord*>`	O(1) push / pop	Enforces First-In, First-Out (FIFO) triage order in the consultation waiting room.
Stack	`std::stack<AdminAction>`	O(1) push / pop	Powers the administrative **Undo** engine (LIFO). Accidental record deletions or modifications are popped and reversed in reverse chronological order.
Array	`PatientRecord* recentPatients[10]`	O(1) indexed access	A fixed-size 10-slot cache storing the Most Recently Accessed records, reducing database scan frequency for active patients. The array is held from the oldest record at the front to the newest at the back; a record that is used again moves to the newest place, and when the cache is full the record that was used the longest time ago is the one that is dropped.

4. Manual Algorithm Implementation

The brief rules out the STL algorithms, so every search and sort below is written by hand:

4.1 Searching Algorithms

1. **Linear Search (O(N)):** Implemented in `SearchEngine::linearSearchByText`. Scans arbitrary, unsorted rosters element-by-element. It matches the identifier **and** the name, without regard to letter case, and a part of either is enough - so the staff can type "chi" and find "Chioma Eze". It returns **every** match, not only the first one.
2. **Binary Search (O(log N)):** Implemented in `SearchEngine::binarySearchByID`. Operates on sorted rosters by calculating the midpoint `mid = low + (high - low) / 2` and halving the search space each cycle. The GUI makes a sorted copy of the records before calling it, so the precondition always holds.
3. **Recursive Search:** Implemented in `SearchEngine::recursiveSearchByName`. Solves patient lookup by recursively advancing an index pointer until hitting either base case (match located or end of array reached). It collects every patient whose name holds the text.
4. **Category Search:** Implemented in `SearchEngine::searchByCategory`. Returns every record of a category, which is what the clinic staff needs when they want to see, for example, all emergency cases.

4.2 Sorting Algorithms

1. **Selection Sort (O(N^2)):** Iteratively identifies the minimum element in the unsorted partition and performs a single swap per outer iteration. Highly suitable for small datasets where write operations must be minimized.
2. **Merge Sort (O(N log N)):** A divide-and-conquer algorithm that recursively bisects the patient vector into singletons and merges them into sorted temporary buffers. Guarantees optimal O(N log N) execution regardless of initial data distribution. The merge takes the record of the left half first when both records have the same key, so the sort is **stable** and equal records keep the order they had.
3. **Multi-Field Comparator:** Sorting algorithms accept an abstract criteria parameter (`SortField::PatientID` , `SortField::PatientName` , `SortField::PatientAge`), permitting dynamic user-driven reordering from the GUI.
4. **Case-Insensitive Comparison:** The name and the identifier are compared without regard to letter case, using the same rule as the search. A name that was typed in small letters therefore sits in its proper place in the list instead of being pushed to the end by the ASCII order, and the binary search keeps working, because it uses the very same comparison that the sort uses.

5. Graphical User Interface (GUI) & Event Architecture

The graphical frontend was built natively using **Qt 6.5.3 (C++17)**: - **Layout Management (Req. 8):** Hardcoded pixel coordinates were strictly avoided. Responsive layouts adapt dynamically to window resizing using `QVBoxLayout` , `QHBoxLayout` , `QFormLayout` and `QGridLayout` . The forms use `QFormLayout` , so a label and its field stay together and the form grows with its longest label; the panels use `QHBoxLayout` and `QVBoxLayout` , so the columns and the rows share the space that is left; and the three tables of figures and of categories and priorities use `QGridLayout` , so the entries line up under each other in both directions. - **Event-Driven Programming (Req. 6):** User actions trigger Qt Signals connected to C++ Member Slots. - *Button events (6.a):* Add, Edit, Delete, Undo, Book, Mark as Completed, Cancel, Search, Sort, Save, Load and New Database are all wired to a slot. - *Menu events (6.b):* The File menu holds **New**, **Save as...** (`Ctrl+S`), **Load...** (`Ctrl+O`) and **Exit** (`Ctrl+Q`); the Edit menu holds **Undo last operation** (`Ctrl+Z`) and **Quick search** (`Ctrl+F`); the Help menu holds **About**. - *Text field events (6.c):* the appointment identifier field records that the staff typed it, so that the free identifier is not written over their work; and the patient identifier field answers while it is being typed, telling the staff whether the identifier is free or already in the clinic. - *Timer events (6.d):* a one-second `QTimer` keeps the clock and the live numbers of the clinic in the status bar, shows a reminder of the bookings that are still pending, and refreshes the statistics of the dashboard. - **Multi-Panel Navigation (Req. 7):** Organized via a central `QTabWidget` containing: - *Panel 1:* Patient Registration form and color-coded Roster table (`gui/PatientPanel.cpp`). - *Panel 2:* Appointment booking form and schedule grid (`gui/AppointmentPanel.cpp`). - *Panel 3:* Waiting room line manager with FIFO calling dialogs (`gui/QueuePanel.cpp`). - *Panel 4:* Search and sort engine with real-time algorithm selection dropdowns (`gui/SearchPanel.cpp`). - *Panel 5:* Clinic Reports generator (`gui/ReportPanel.cpp`). - *Panel 6:* Administration panel with the administrative steps, the live dashboard of the clinic and the recently accessed records (`gui/AdminPanel.cpp`). - **Separation of concerns (Req. 2):** Every tab is its own panel class that owns its widgets and talks to the `ClinicManager` , never to the database. `gui/MainWindow.cpp` only builds the tabs, the menu bar and the status bar, and it refreshes the panels when one of them reports a change. - **Advanced GUI Techniques (Req. 9):** - Custom table cell presentation using soft colors (`#ffebee` for Emergency, `#e8f5e9` for General, `#e3f2fd` for Follow-Up), with a table of the categories and their priorities beside the roster and beside the dashboard. - Fields that appear and disappear with the selected patient type. - Background `QTimer` maintaining live clock synchronization, queue size updates, a reminder of the pending appointments and a live refresh of the dashboard statistics. - Standard file dialog integration via `QFileDialog` for custom file import/export. - Accessible keyboard shortcuts (`Ctrl+N` , `Ctrl+S` , `Ctrl+O` , `Ctrl+Z` , `Ctrl+F` , `Ctrl+Q`). - A live hint under the patient identifier field that says whether the identifier can be used. - A tooltip on every button, every field and every list, which says what the control does in one short sentence.

6. Exception Handling

A custom exception class hierarchy inheriting from standard `std::exception` guarantees error tolerance without abrupt application termination: - `ClinicException`: Base exception class providing overridden `what() const noexcept`. - `DuplicateIDException`: Thrown when attempting to register an ID already present in the database. - `InvalidPatientDataException`: Thrown for illegal ages (< 0 or > 120) or blank mandatory name fields. - `InvalidAppointmentException`: Thrown for an appointment without an ID, patient, date or time, for a duplicated appointment ID and for a patient that is not registered. - `FileIOException`: Thrown during file stream access failures.

All GUI event slots wrap database mutations inside `try { ... } catch (const ClinicException& e) { ... }` blocks, presenting descriptive `QMessageBox::critical` alerts to users. Rows in a data file that cannot be read are skipped and listed for the user instead of silently turning into empty records.

The safety net. Qt does not catch an exception that leaves a slot: it travels out of the event loop and ends the program with "*terminate called after throwing an instance of ...*". Because requirement 11 asks for a program that keeps running after a normal mistake, `gui_main.cpp` uses a small `QApplication` subclass whose `notify()` method catches anything that escapes an event and shows it to the staff as a message. The rules of the clinic are checked in the controller layer, so the panels usually never let an exception through, but the program still has a last line of defence.

The rules are checked on every way in. A record is checked when it is registered, when it is edited and when it is read from a file. The contact number rule, for example, was at first only applied on registration, so an edit could put an illegal contact number into a record. The edit path now runs the same `ClinicManager::validatePatient` check as the registration path.

7. Challenges and Fixes

1. **One Class That Did Too Much:**
2. *Challenge:* The graphical panels called the database directly, so every rule of the clinic - a valid age, a date written correctly, a patient that may be booked - ended up inside `ClinicDatabase`, which grew to 488 lines and mixed the data handling with the business logic.
3. *Resolution:* The rules were moved up into `ClinicManager` and `AppointmentController` in the controller layer, and the panels were repointed from the database to the manager. `ClinicDatabase` now only stores and finds.
4. **The Binary Search Showed The Wrong Patient:**
5. *Challenge:* The panel made a sorted copy of the records and searched in that copy. The routine returned the **place** of the record it found, and the panel then used that place to look the record up in the unsorted roster, so it showed a patient that had nothing to do with the search.
6. *Resolution:* The search returns the records that matched, not their positions, so the record shown is always the record that was found.
7. **The Cache Of Frequently Accessed Records Did Not Work:**
8. *Challenge:* The fixed-size array of ten records was written with three different ideas of the order at the same time, so it kept the order in which the records were *registered* rather than the order in which they were *used*. A cache hit did not even record the access.
9. *Resolution:* One order is used everywhere now: the array is held from the oldest record at the front to the newest at the back, using a record moves it to the newest place, and a new record is added at the back.
10. **A Failed Save Could Destroy The Data That Was Already Saved:**

11. *Challenge:* The file was opened for writing straight away, and an ofstream empties the file the moment it is opened. If the writing then failed, a half written file was left behind and the program still said "The data was written".
12. *Resolution:* The records are written to a temporary file first, and the real file is only replaced when the writing has finished, so a failed save leaves the old file untouched.
13. **Loading A File Could Silently Throw The Appointments Away:**
14. *Challenge:* The save asked for two file names, but the load asked for one and then looked for a file called "appointments.csv" next to the patient file. Appointments saved under any other name were silently lost, and loading emptied the whole clinic without asking.
15. *Resolution:* The load asks for the appointment file as well, offers the one next to the patient file when the staff cancels, and asks before the clinic is replaced.
16. **One Mistake Could End The Whole Program:**
17. *Challenge:* Qt does not catch an exception that leaves a slot. The exception travels out of the event loop and the program ends with "terminate called after throwing an instance of ...", and the windows disappear.
18. *Resolution:* The application class in gui_main.cpp overrides `notify()`, the one place every event passes through, so anything that escapes a slot is caught and shown as a message.

8. Conclusion

The Smart Clinic Appointment & Patient Management System is a working C++ application that covers the lab requirements: object-oriented design, manual algorithms, the required data structures, and a Qt interface.